Deep-Based Object Detection - Part 2

Jefersson A. dos Santos
j.santos@sheffield.ac.uk

Road Map

Previous lectures:

- Introduction to Deep Learning
  - Basics
  - Training Process
- Convolutional Neural Networks
  - Convolutional Layers
  - Data Augmentation
  - Fine Tuning
- Modern CNN Architectures
  - VGG, ResNet, DenseNet, etc

Last lecture:

- Object Detection
- Single-Stage Architectures
  - YOLO

Today's lectures:

- Double-Stage Architectures
  - R-CNN Family
- Semantic Segmentation

Road Map

Previous lectures:

- Introduction to Deep Learning
  - Basics
  - Training Process
- Convolutional Neural Networks
  - Convolutional Layers
  - Data Augmentation
  - Fine Tuning
- Modern CNN Architectures
  - VGG, ResNet, DenseNet, etc

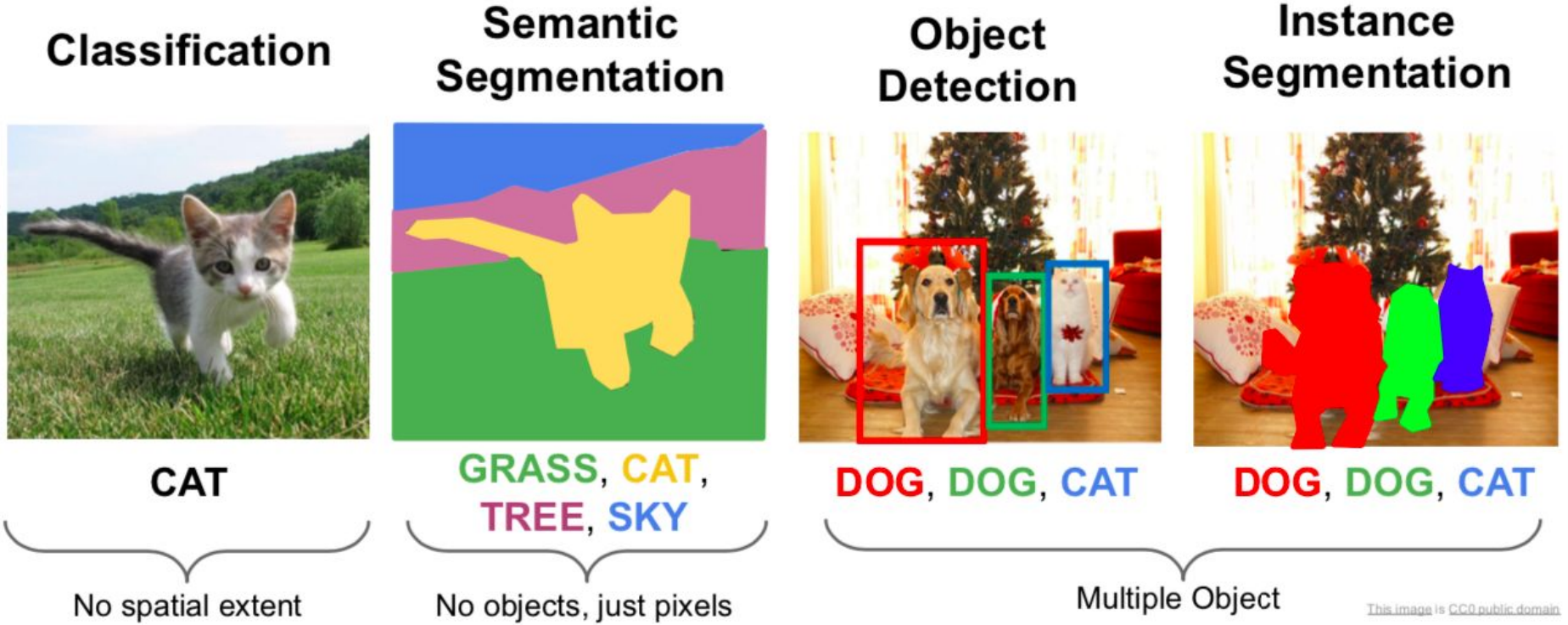
Last lecture:

- Object Detection
- Single-Stage Architectures
  - YOLO

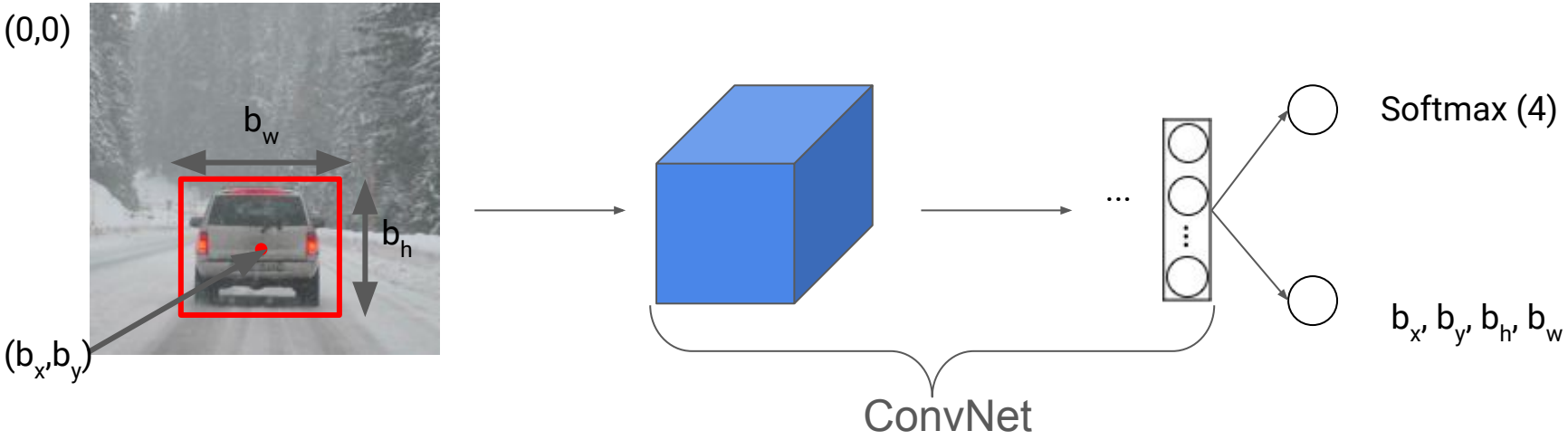
Today's lectures:

- **Double-Stage Architectures**
  - **R-CNN Family**
- Semantic Segmentation

Computer Vision Tasks



Example: classification with localization



Classes

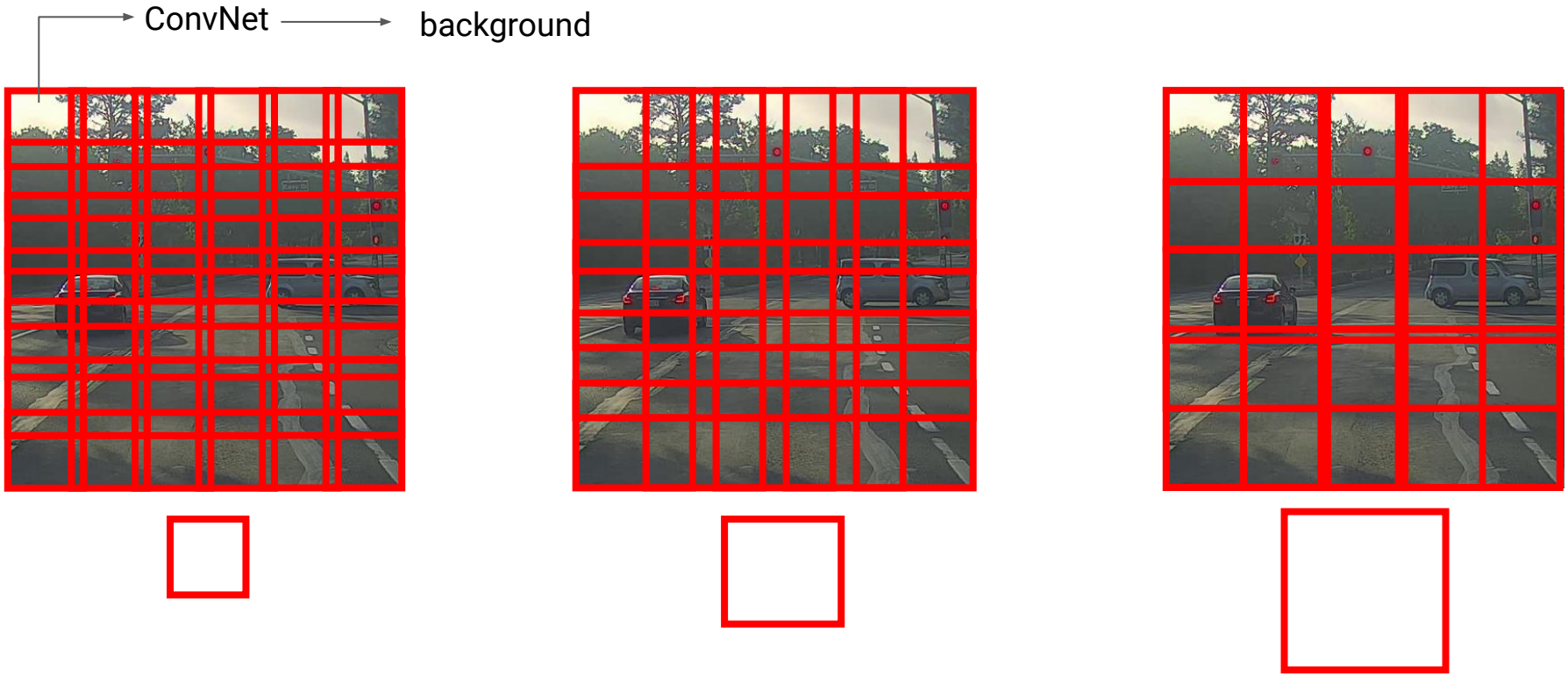
1. Pedestrian
2. Car
3. Motorcycle
4. Background

Bounding box

$b_x = 0.5$
$b_y = 0.7$
$b_h = 0.3$
$b_w = 0.4$

Location is modeled as a regression problem

Sliding windows detection

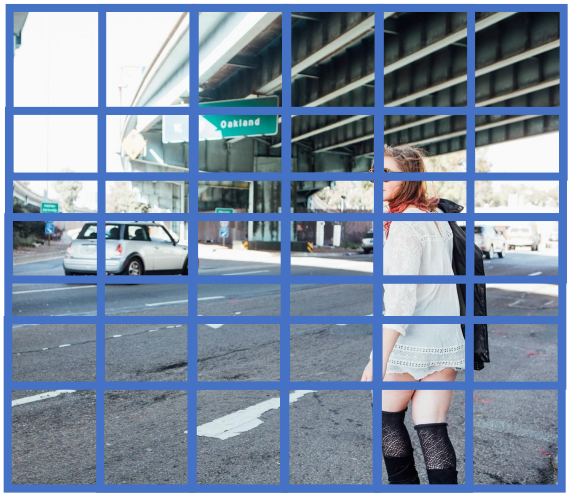


Detection by sliding windows - Summary

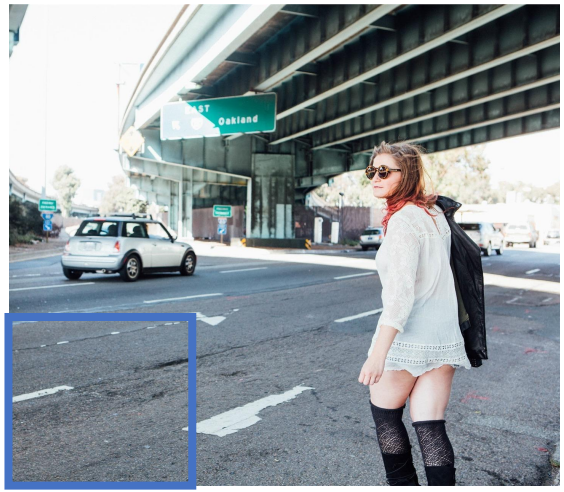
- It starts with a small window that slides from left to right, top to bottom, according to the chosen stride. For each window, we use ConvNet to return a prediction.
- The same algorithm starts again, but with larger windows.
- Problem: computational cost
  - We can increase stride, but performance may drop
- Alternative solutions:
  - not classify all regions (region proposal) → R-CNN Family! (two stage)
  - convolutional implementation of sliding windows → YOLO! (single stage)

Object Detection with Region Proposals

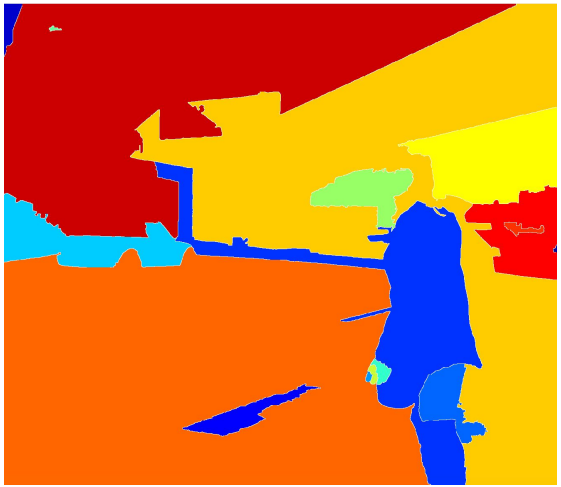
Region proposal: R-CNN



Slide window to classify each image block



Inefficient: too many regions with nothing relevant to classify



R-CNN: it selects only some windows

Segmentation algorithm: ~2000 *blobs*

[Girshik et. al, 2013, Rich feature hierarchies for accurate object detection and semantic segmentation]

Region proposal: R-CNN

In a sense, object detection is inefficient: most regions do not contain objects

- R-CNN: finds "blobs" (regions of interest) and classifies only them
  - Uses selective search (segmentation algorithm) to find edges that separate regions of the image (returns ~2000 blobs)
  - Particularly efficient when objects have many different shapes/scales

Selective Search Algorithm

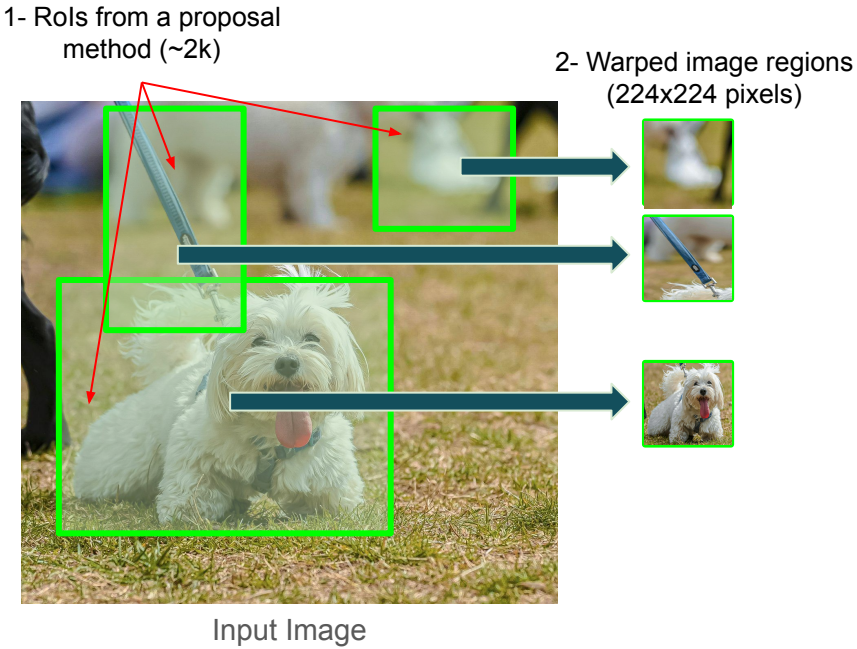
1. Generate initial sub-segmentation, we generate many candidate regions
2. Use greedy algorithm to recursively combine similar regions into larger ones
3. Use the generated regions to produce the final candidate region proposals

R-CNN Pipeline

Input Image

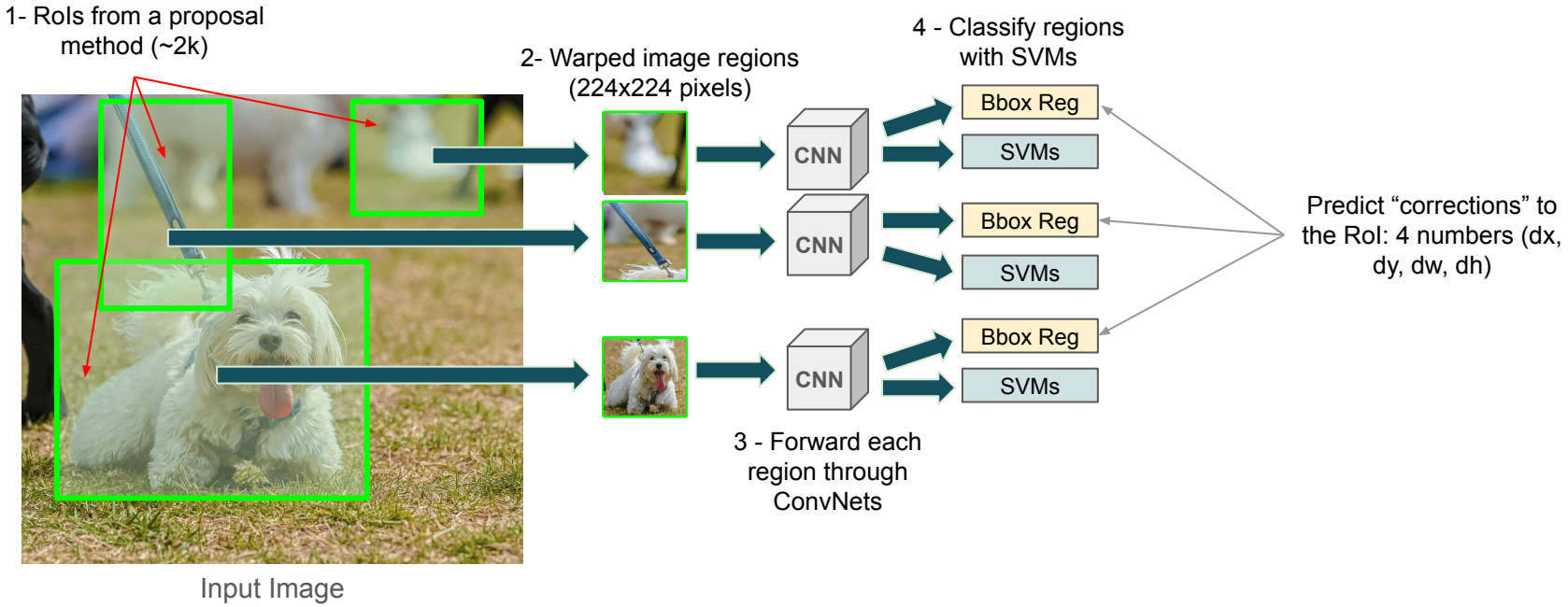
[Girshik et. al, 2013, Rich feature hierarchies for accurate object detection and semantic segmentation]

R-CNN Pipeline



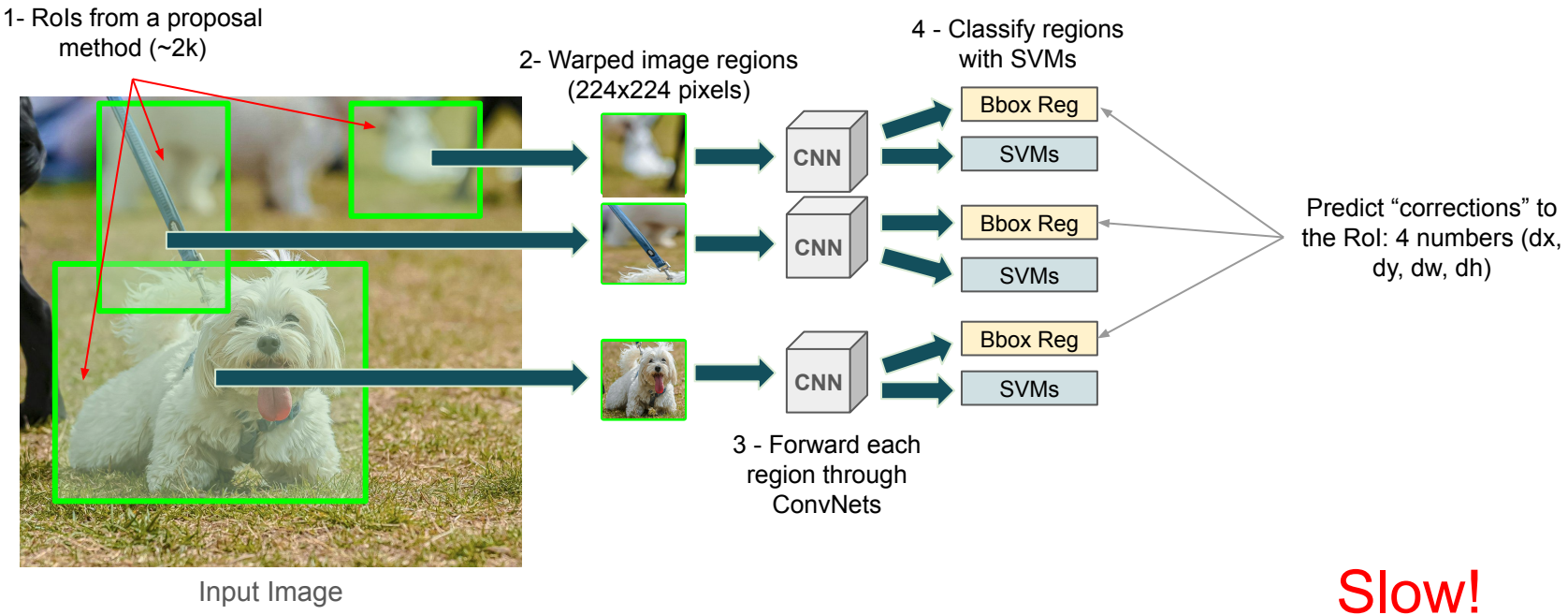
[Girshik et. al, 2013, Rich feature hierarchies for accurate object detection and semantic segmentation]

R-CNN Pipeline



[Girshik et. al, 2013, Rich feature hierarchies for accurate object detection and semantic segmentation]

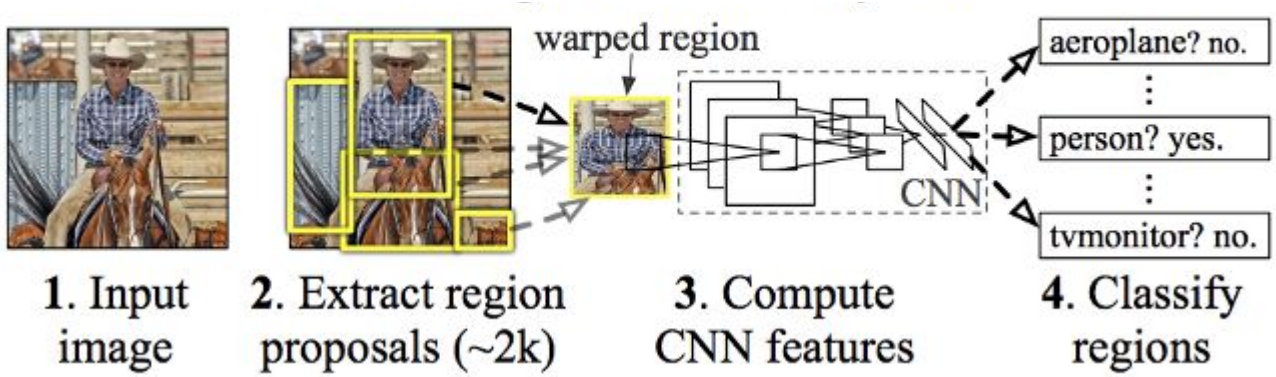
R-CNN Pipeline



Slow!

[Girshik et. al, 2013, Rich feature hierarchies for accurate object detection and semantic segmentation]

R-CNN - Summary



RoI (regions of interest) may be "wrong"; returns bboxes too

- Select Region of Interests (RoIs) with an heuristic-based algorithm
- Use **pre-trained CNN to extract features** from each RoI by using a forward computaction
- Train a SVM to obtain the class scores
- Train linear regression to predict *bounding boxes*

R-CNN Training - Bounding Box Regression

We want to train linear regression using CNN output from proposed region

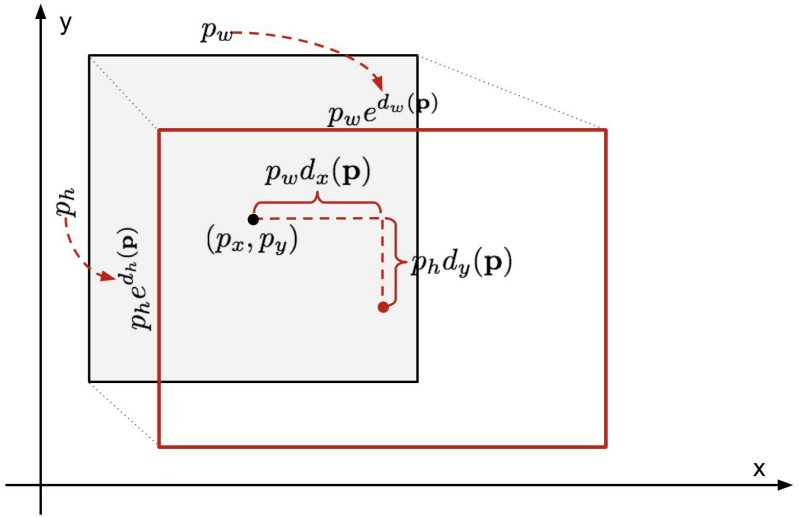
Input: N pairs $\{(P^{(i)}, G^{(i)})\}_{i=1,...,N}$, where

$P^{(i)} = \{P_x^{(i)}, P_y^{(i)}, P_h^{(i)}, P_w^{(i)}\}$ are the coordinates of proposed region and

$G^{(i)} = \{G_x^{(i)}, G_y^{(i)}, G_h^{(i)}, G_w^{(i)}\}$ are the coordinates for the ground truth bbox.

Linear regression is a transformation that maps P to G, which is parameterized by

- $d_x(P)$ e $d_y(P)$: scale invariant translation of the center of the bbox P
- $d_h(P)$ e $d_w(P)$: log scale transformations of the height and width of P



R-CNN Training - Bounding Box Regression

Interrelationship between P e G:

$$\hat{G}_x = P_w d_x(P) + P_x$$
$$\hat{G}_y = P_h d_y(P) + P_y$$
$$\hat{G}_w = P_w \exp(d_w(P))$$
$$\hat{G}_h = P_h \exp(d_h(P)).$$

Learnable parameters!

Targets for $d_*(P)$:

$$t_x = (G_x - P_x)/P_w$$
$$t_y = (G_y - P_y)/P_h$$
$$t_w = \log(G_w/P_w)$$
$$t_h = \log(G_h/P_h).$$

Função a ser otimizada:

$$\mathcal{L}_{\text{reg}} = \sum_{i \in \{x,y,w,h\}} (t_i - d_i(\mathbf{p}))^2 + \lambda \|\mathbf{w}\|^2$$

R-CNN Limitations

- Training time is high because it needs to classify ~2k RoIs per image
- Selective search is fix (not learnable) and can select bad regions
- It cannot be implemented in real time since it takes 47s for each test image (~2k independent forward passes per image)

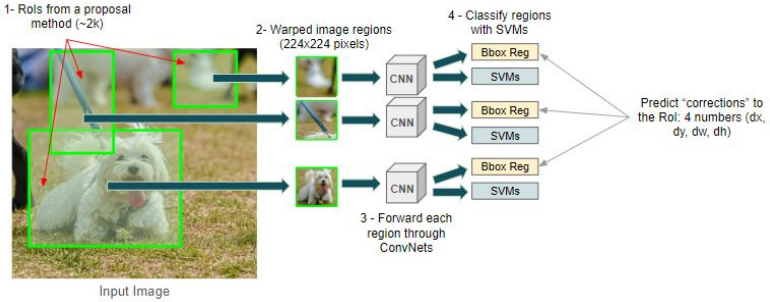
Idea: processing images before cropping! Change order between ConvNet and cropping.

Object Detection with Region Proposals

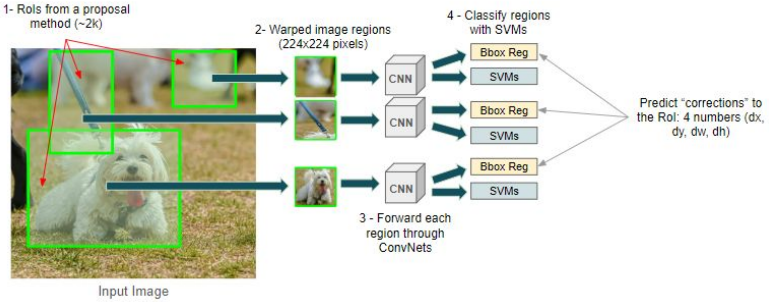
Fast R-CNN

Fast R-CNN

Input Image



Fast R-CNN



Input Image

2- RoIs from a proposal method

CNN

1- Run whole image through a pre-trained network (**Backbone**)

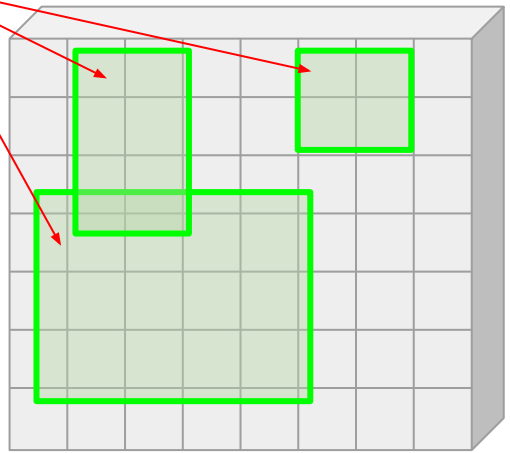
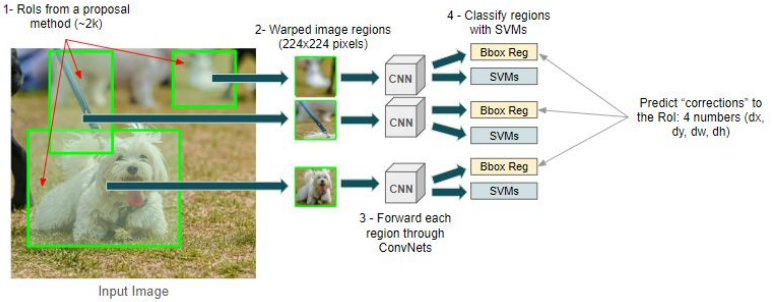


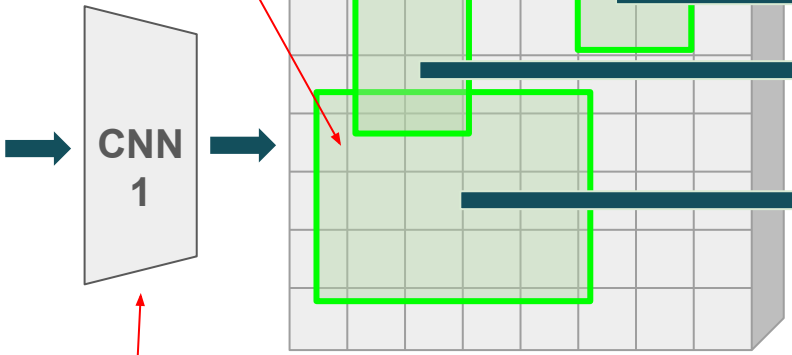
Image Features

Fast R-CNN



Input Image

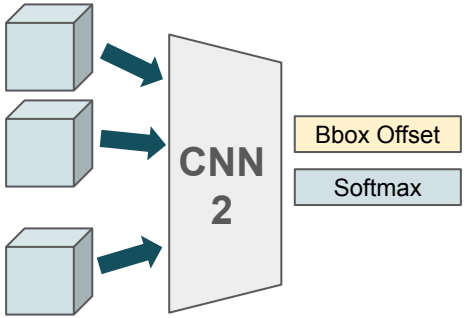
2- RoIs from a proposal method



1- Run whole image through a pre-trained network (**Backbone**)

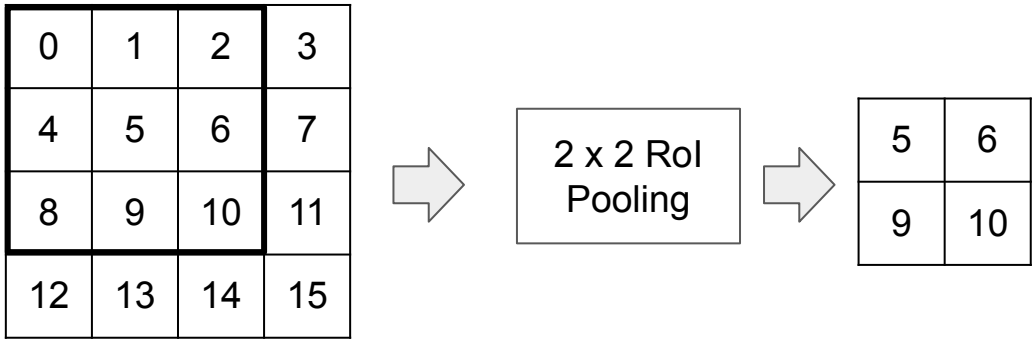
Image Features

4 - Per-Region Network



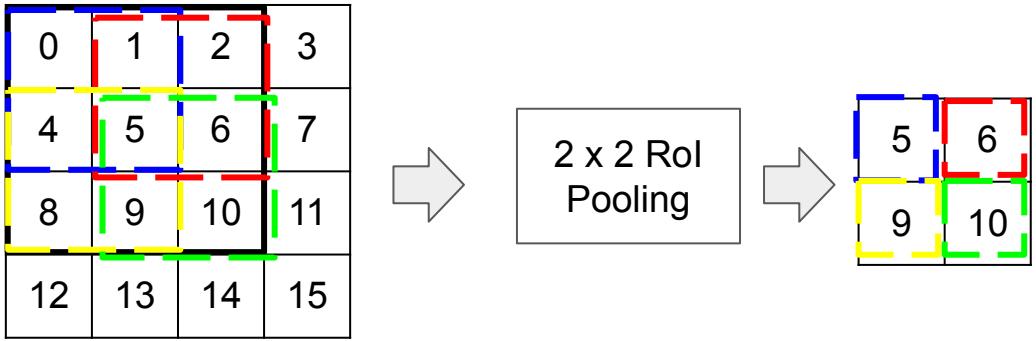
3 - Crop + Warped features (RoI pooling)

RoI pooling



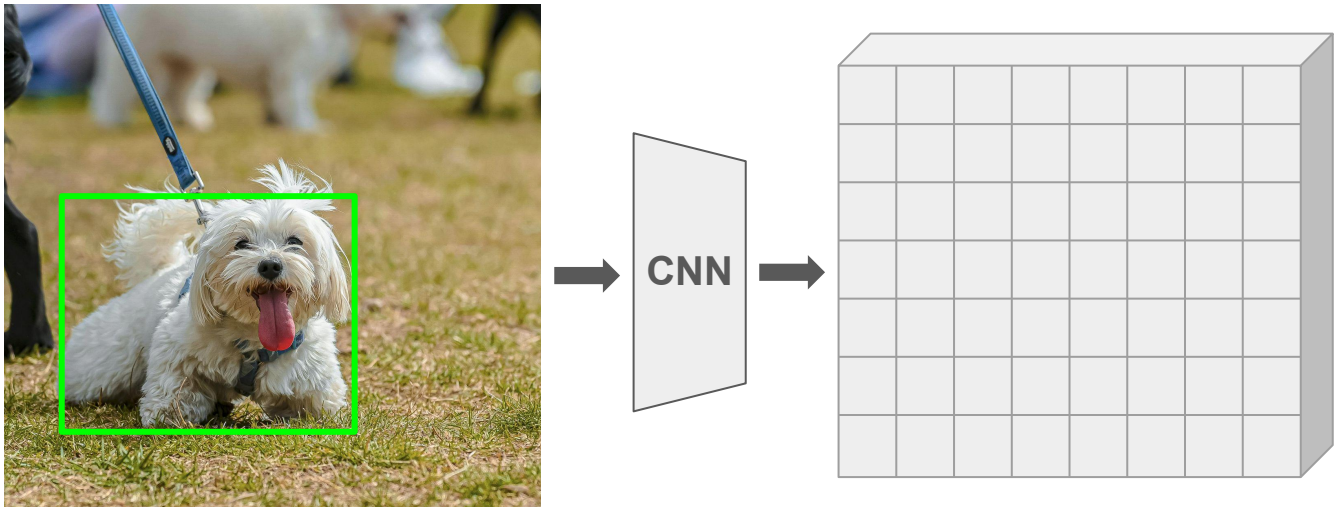
- Given an anchor box (in fig., 3x3 region), it cuts the region uniformly into n x n blocks (in fig., n=m=2) and returns the maximum value of each region
- If RoI has size a x a, window has size $\lceil a/n \rceil$ and stride $\lfloor a/n \rfloor$
- Return n^2 values for each RoI

RoI pooling



- Given an anchor box (in fig., 3x3 region), it cuts the region uniformly into n x n blocks (in fig., n=m=2) and returns the maximum value of each region
- If RoI has size a x a, window has size $\lceil a/n \rceil$ and stride $\lfloor a/n \rfloor$
- Return n^2 values for each RoI

Cropping Features: RoI Pooling

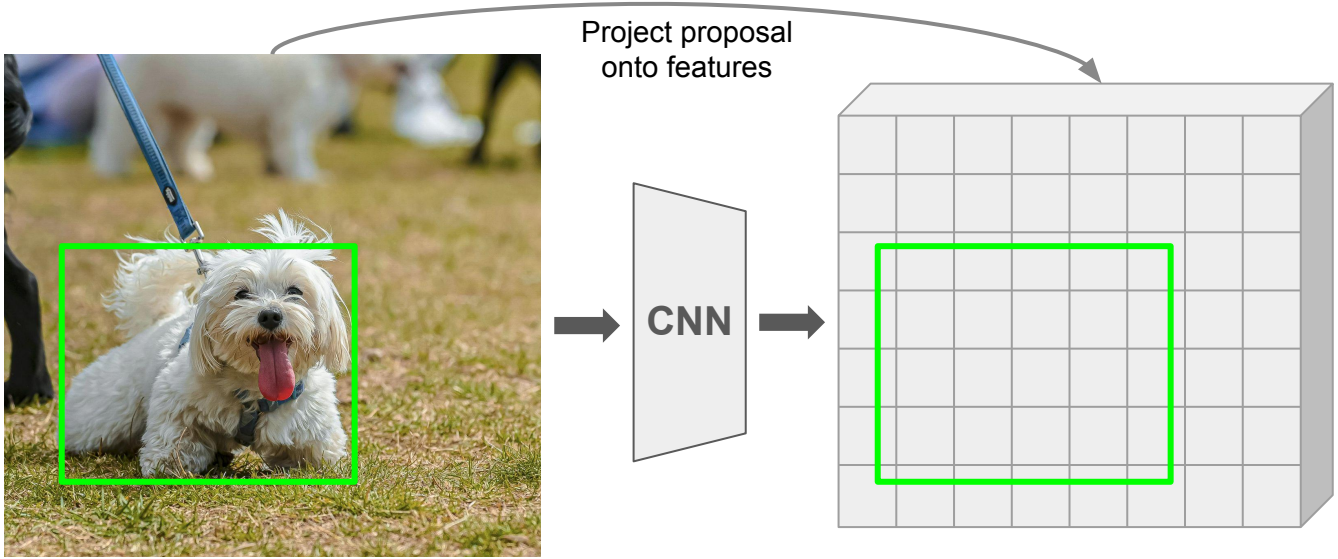


Input Image
(e.g. 3 x 640 x 480)

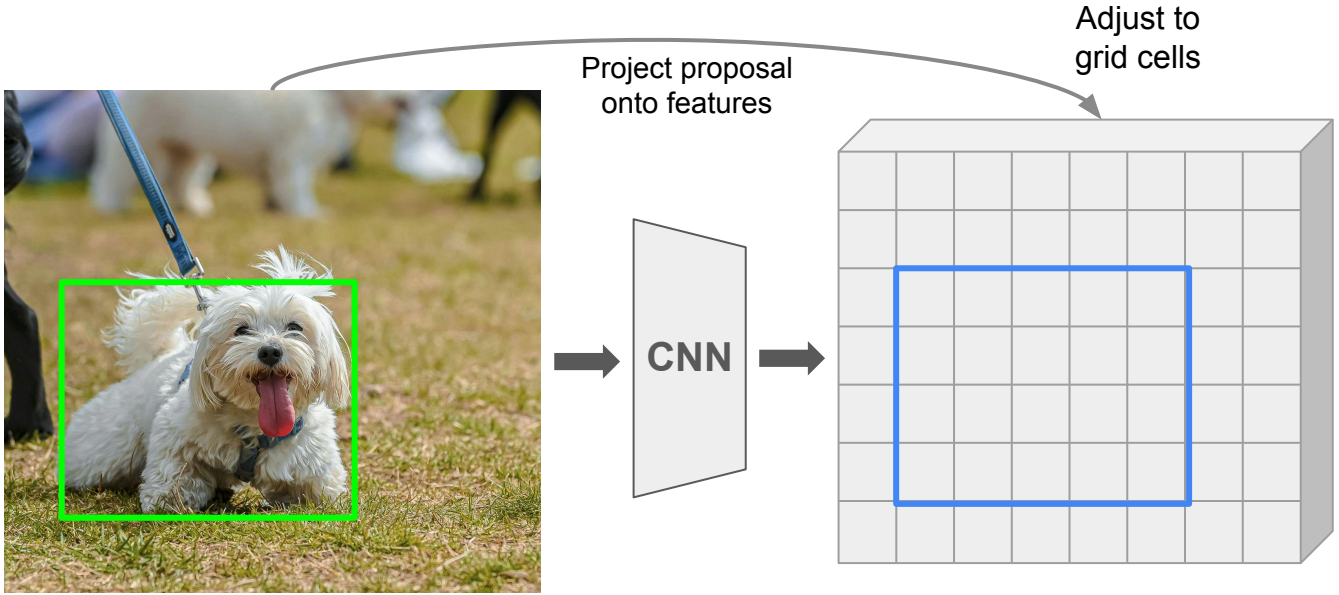
Image Features
(e.g. 512 x 20 x 15)

Adapted from Fei-Fei Li & Justin Johnson & Serena Yeung (Stanford)

Cropping Features: RoI Pooling

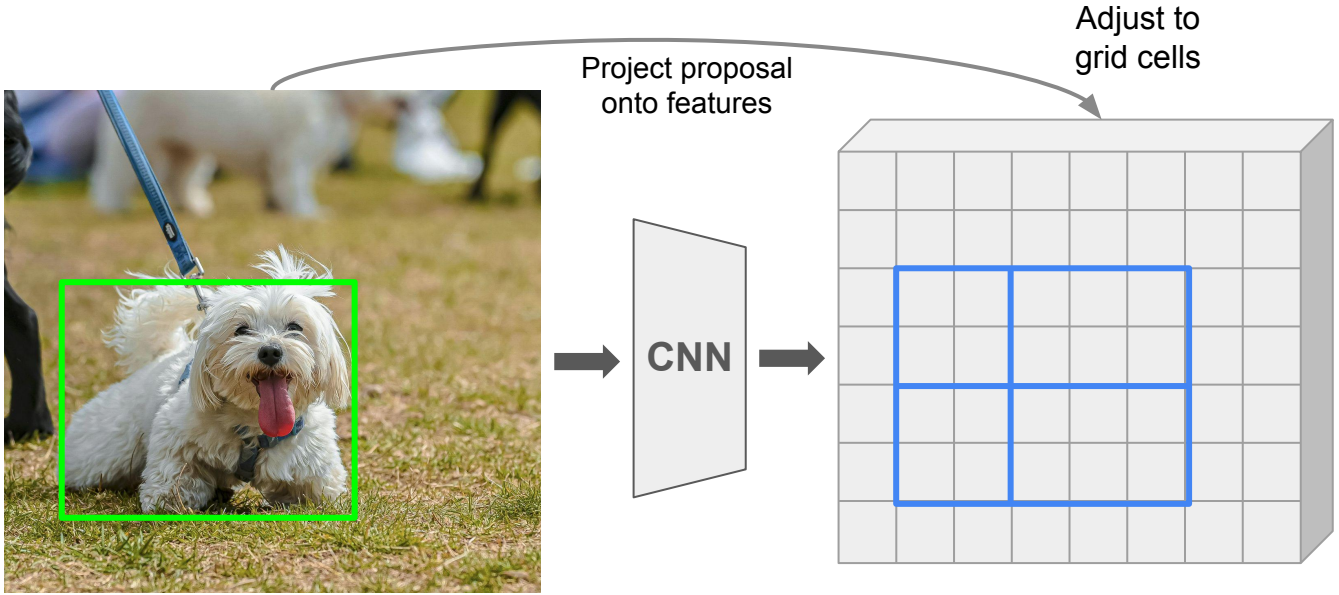


Cropping Features: RoI Pooling

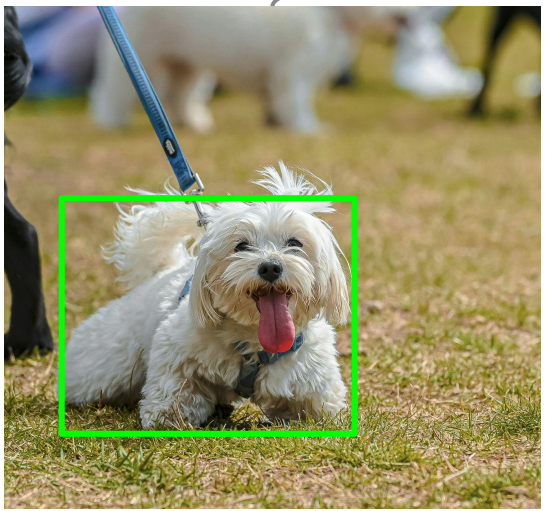


Cropping Features: RoI Pooling

Divide into 2x2 grid of (roughly) equal subregions



Cropping Features: RoI Pooling



Input Image
(e.g. 3 x 640 x 480)

Project proposal
onto features

Adjust to
grid cells

CNN

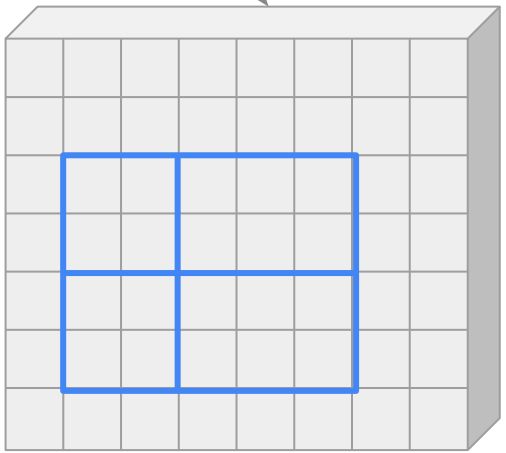
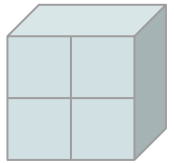


Image Features
(e.g. 512 x 20 x 15)

Divide into 2x2
grid of (roughly)
equal subregions

Max-pool within
Each subregion

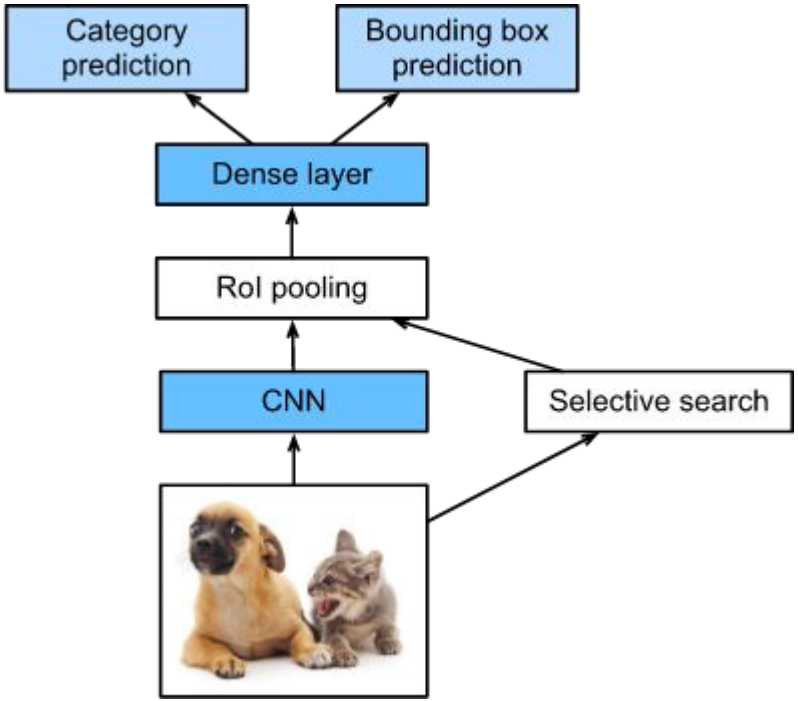


Region Features
(e.g. 512 x 2 x 2)

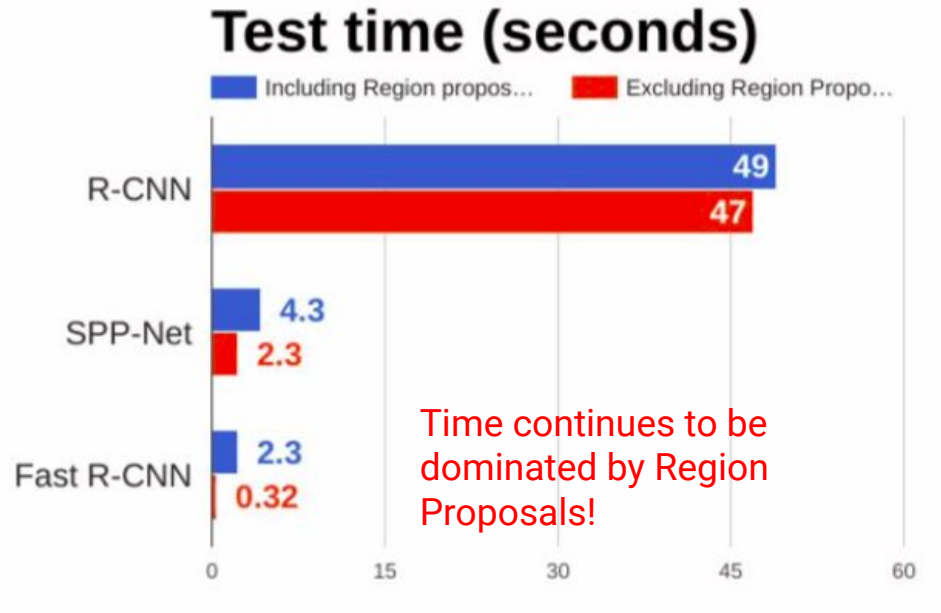
Region features always the
same size even if input regions
have different sizes!

Fast R-CNN

- CNN to extract features from the entire image at once (fast)
- Selective search selects regions (RoIs) from the original image
- RoI (region of interest) pooling takes the features from each region and returns a set of features of a fixed size
- There is an **alignment problem** with RoI Pooling whose solution was only proposed in 2017 (Mask R-CNN)



R-CNN vs. Fast R-CNN



Girshick et al, “Rich feature hierarchies for accurate object detection and semantic segmentation”, CVPR 2014.
He et al, “Spatial pyramid pooling in deep convolutional networks for visual recognition”, ECCV 2014
Girshick, “Fast R-CNN”, ICCV 2015

Object Detection with Region Proposals

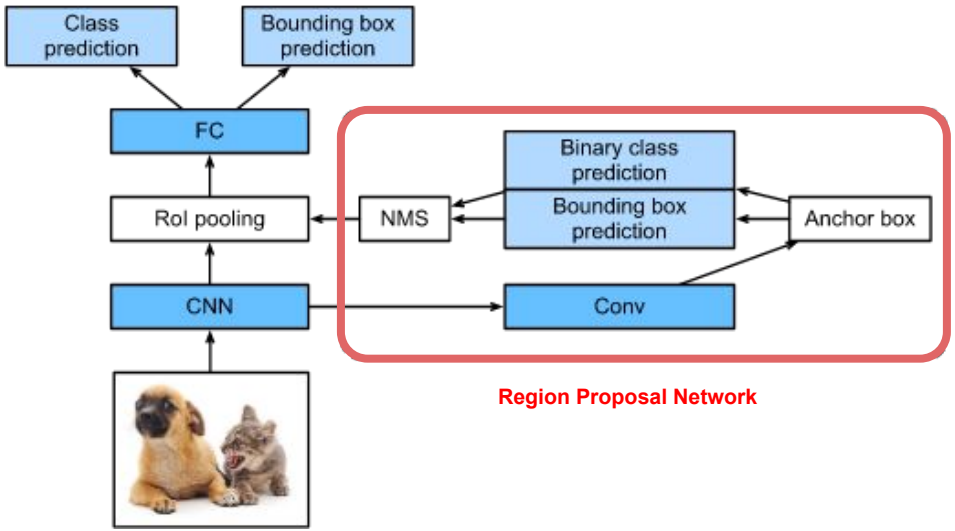
Fast**er** R-CNN

Faster R-CNN

Idea: Make CNN do proposals!

- Add a Region Proposal Network (RPN) to generate proposals based on features.

- The rest remains the same as Fast R-CNN:
  - Extract features for each proposal, then classify each proposal.

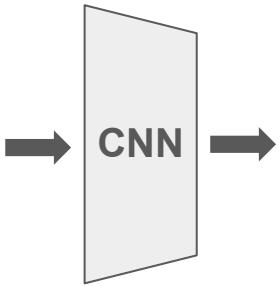


Adapted from: https://d2l.ai/chapter_computer-vision/rcnn.html#r-cnns

Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems* (pp. 91–99).

Region Proposal Network

Input Image
(e.g. 3 x 640 x 480)



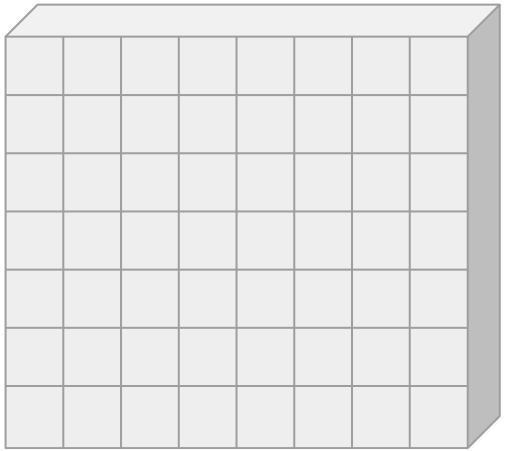
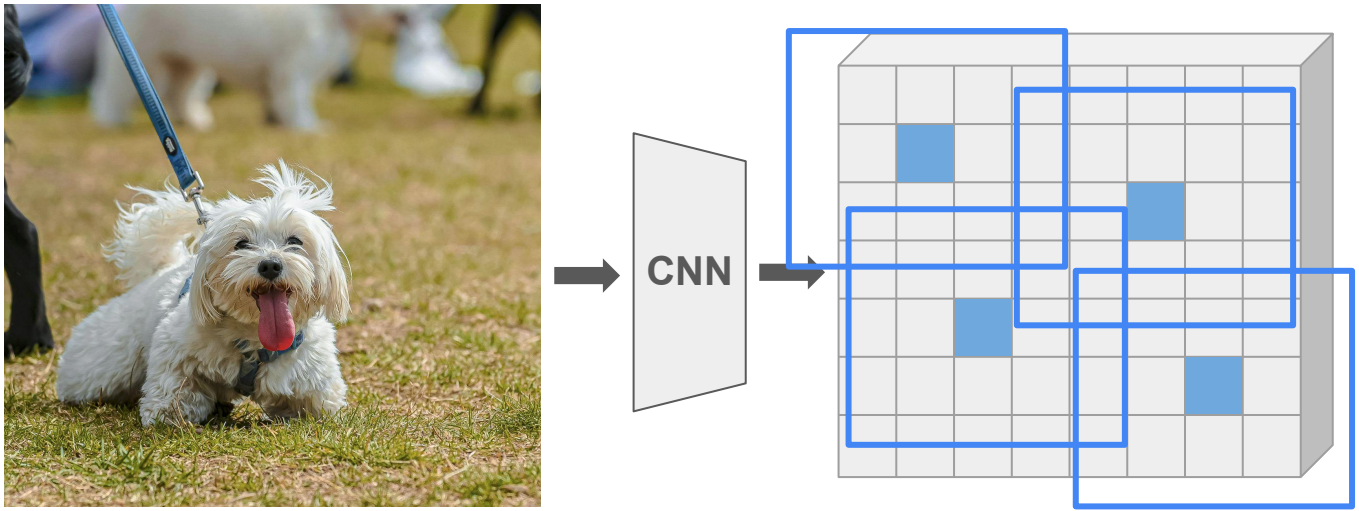


Image Features
(e.g. 512 x 20 x 15)

Adapted from Fei-Fei Li & Justin Johnson & Serena Yeung (Stanford)

Region Proposal Network

Imagine an **anchor box** of fixed size at each point in the feature map



Input Image
(e.g. 3 x 640 x 480)

Image Features
(e.g. 512 x 20 x 15)

Region Proposal Network

Imagine an **anchor box** of fixed size at each point in the feature map

Input Image
(e.g. 3 x 640 x 480)

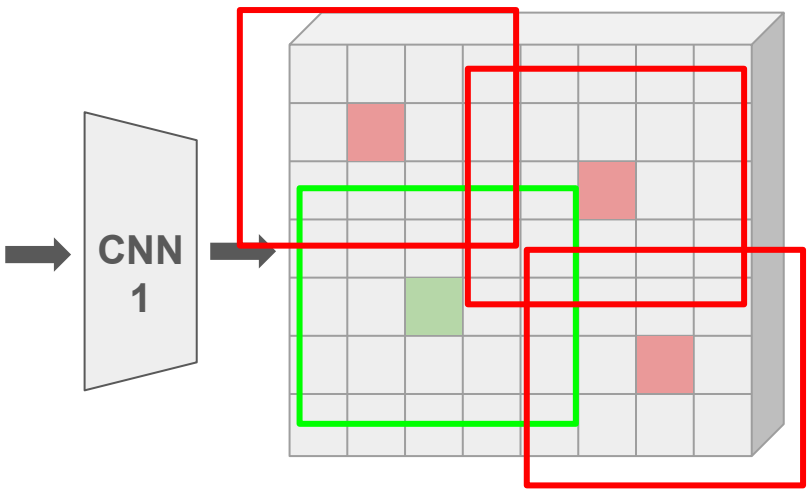
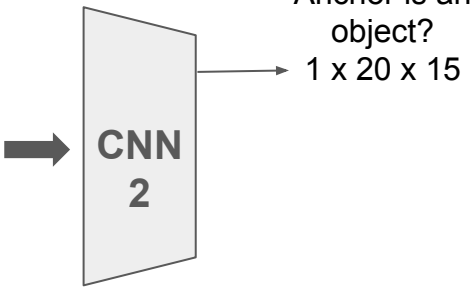


Image Features
(e.g. 512 x 20 x 15)



For every location, determine if the associated anchor encompasses an object.

Region Proposal Network

Imagine an **anchor box** of fixed size at each point in the feature map

Input Image
(e.g. 3 x 640 x 480)

CNN 1

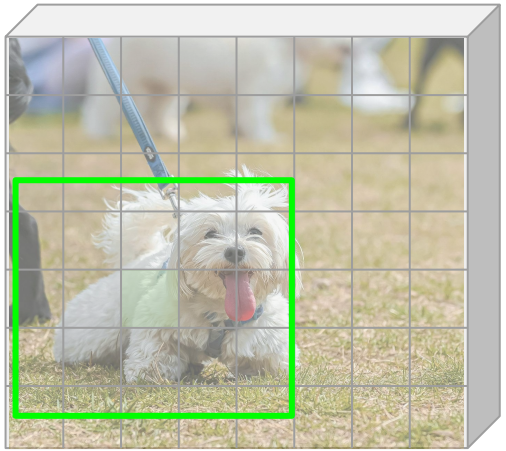


Image Features
(e.g. 512 x 20 x 15)

CNN 2

Anchor is an object?
1 x 20 x 15

Note object is misaligned!

Region Proposal Network

Imagine an **anchor box** of fixed size at each point in the feature map

Input Image
(e.g. 3 x 640 x 480)

CNN 1

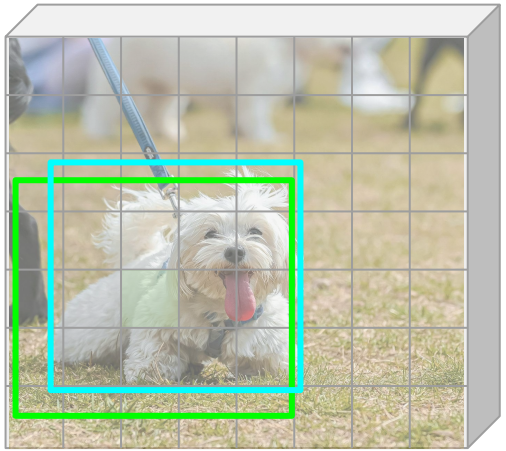


Image Features
(e.g. 512 x 20 x 15)

CNN 2

Anchor is an object?
1 x 20 x 15

Box Transforms
4 x 20 x 15

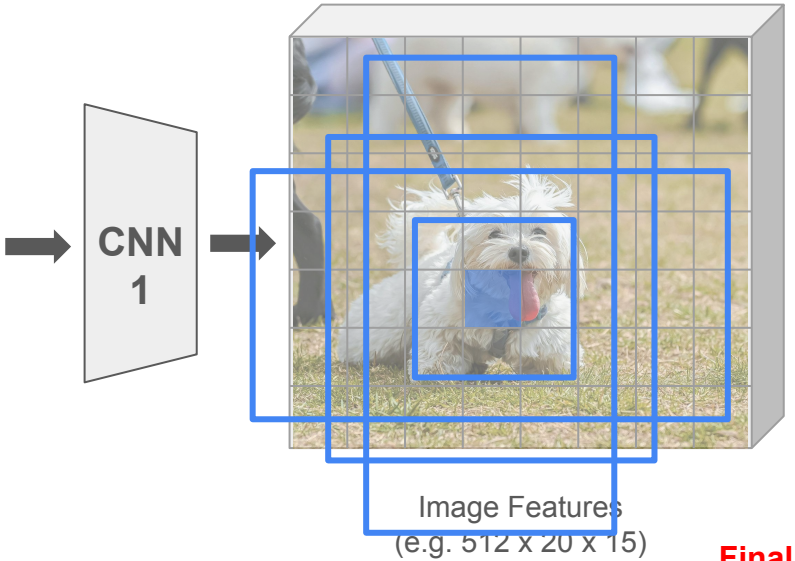
For positive boxes, regression is performed to predict a transformation from the anchor to the ground-truth box, involving four numbers per pixel.

Adapted from Fei-Fei Li & Justin Johnson & Serena Yeung (Stanford)

Region Proposal Network

In practice use **K** different anchor boxes of different size/scale at each point

Input Image
(e.g. 3 x 640 x 480)

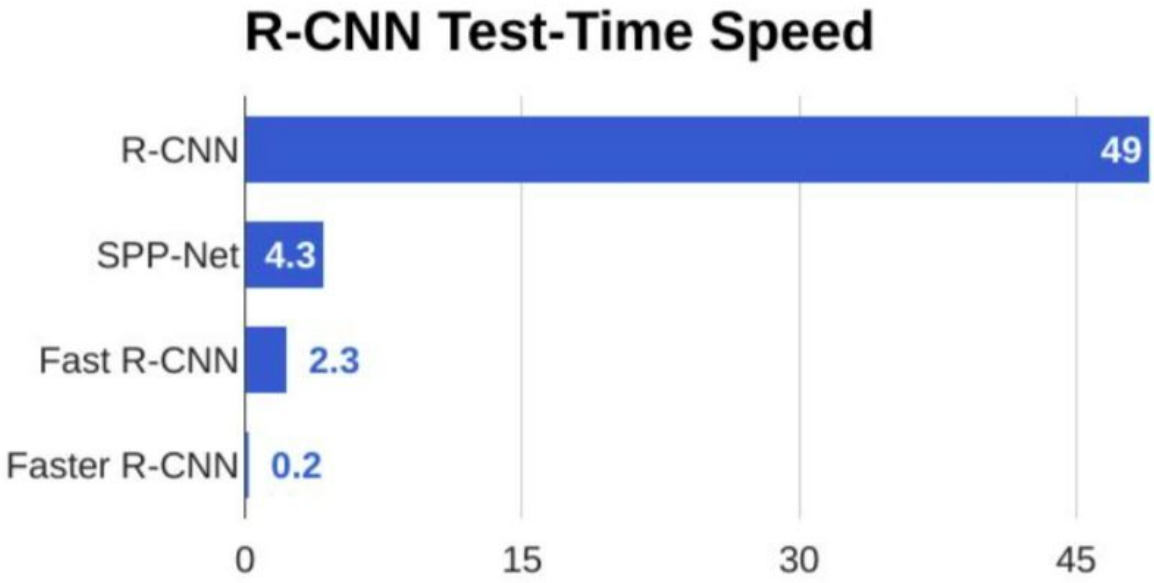


Final Step: Sort the K*20*15 boxes by their "object" score, take top ~300 as our proposals

Adapted from Fei-Fei Li & Justin Johnson & Serena Yeung (Stanford)

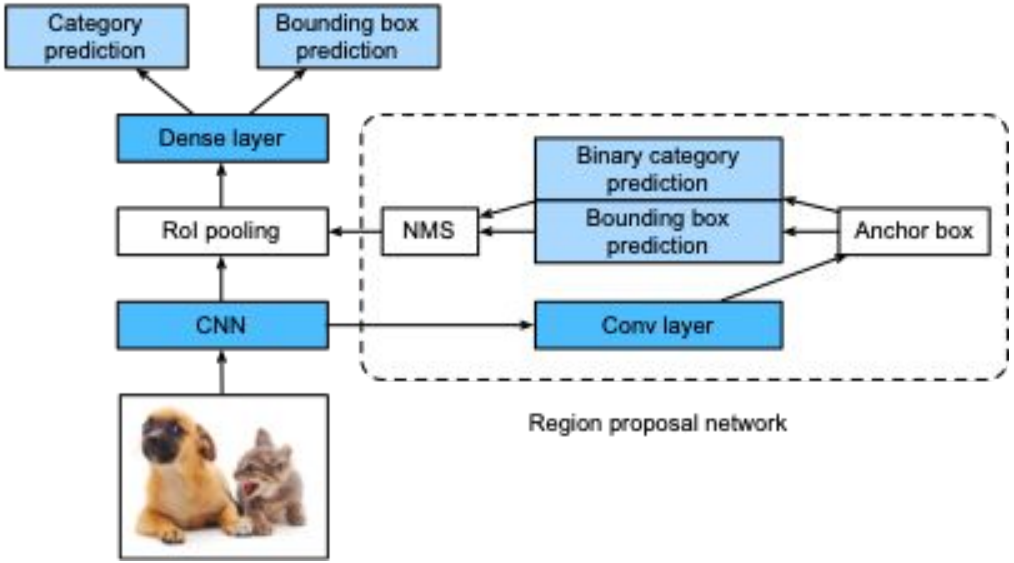
Faster R-CNNs

Idea: make CNNs propose regions!



Faster R-CNN

- Uses a network to propose regions instead of selective search, finding high-quality anchor boxes



R-CNN Family - Summary

R-CNN: Uses external region proposal. Sorts the proposed regions one at a time. Returns class label + bounding box

Fast R-CNN: Uses external region proposal. Uses the convolutional implementation of sliding windows to classify all proposed regions.

Faster R-CNN: Uses convolutional network to propose regions.

[Girshik et. al, 2013, Rich feature hierarchies for accurate object detection and semantic segmentation]
[Girshik, 2015. Fast R-CNN]
[Ren et. al, 2016. Faster R-CNN: Towards real-time object detection with region proposal networks]

Deep-Based Image Segmentation

Jefersson A. dos Santos
j.santos@sheffield.ac.uk

Road Map

Classification

CAT

No spatial extent

Semantic Segmentation

GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection

DOG, DOG, CAT

Instance Segmentation

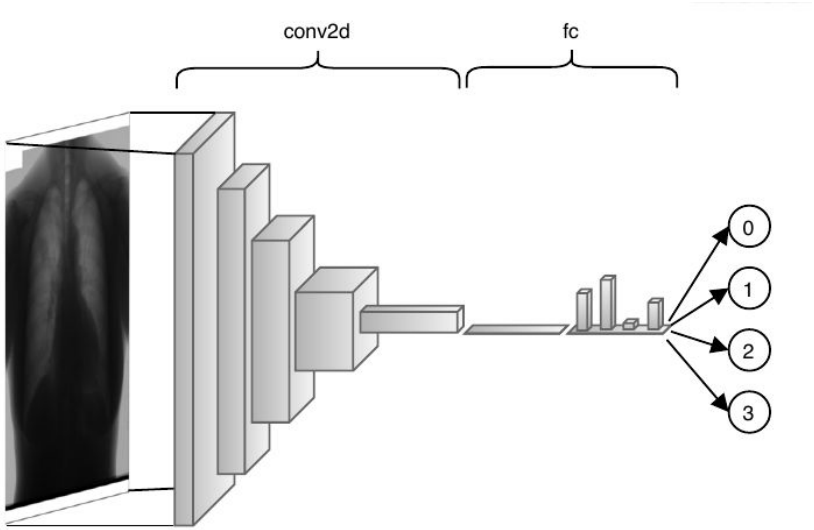
DOG, DOG, CAT

Multiple Object

Classification → Sparse Labelling

Classification is a sparse prediction problem. We want to assign a label to an image.

- CNNs are the most common architectures



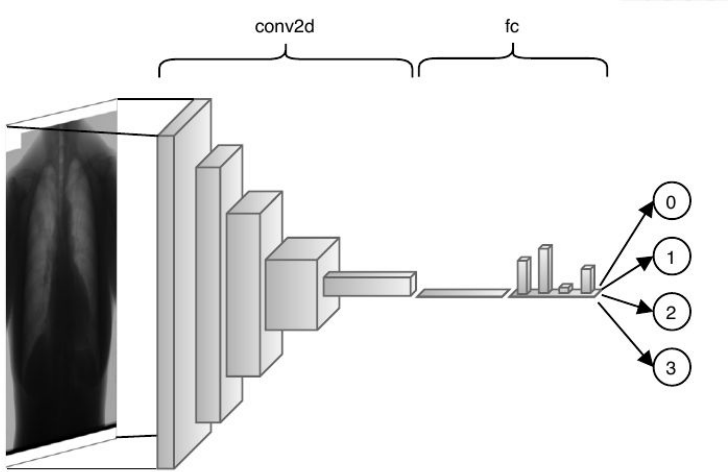
Classification → Sparse Labelling

Main layers:

- Convolutional
- Pooling
- Fully-Connected

Other layers/functions:

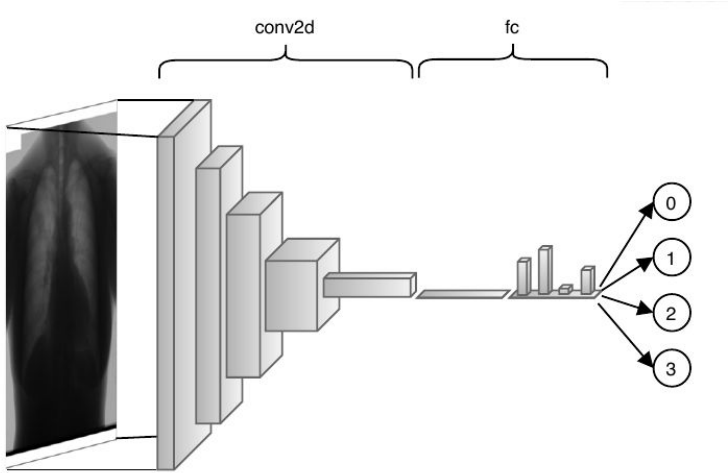
- Rectified Linear Unit (ReLU)
- Batch normalization
- Softmax – for classification
- Sigmoid – for regression



Dense Labelling X Sparse Labelling

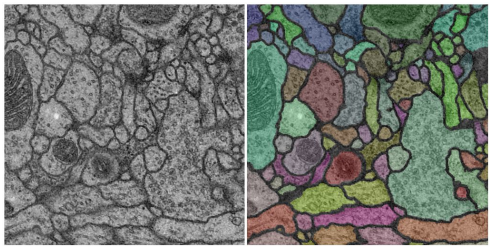
- Sparse labelling
  - Object classification
  - Regression of the probability of malignancy in radiological images
  - Classification between crop images and natural vegetation aerial images

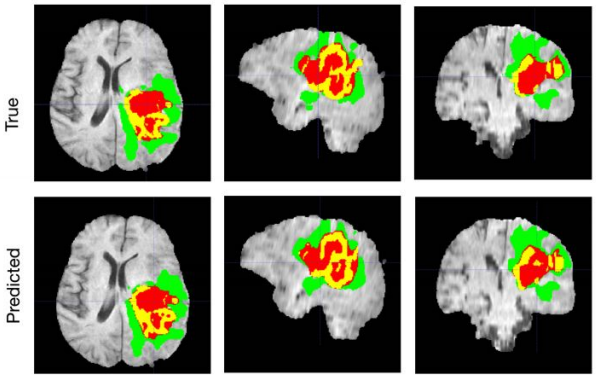
- Dense labelling
  - Object segmentation
  - Tumor segmentation
  - Crop segmentation



Applications

- Medical diagnosis
- Geographic Mapping
- Geology
- Autonomous driving
- ...

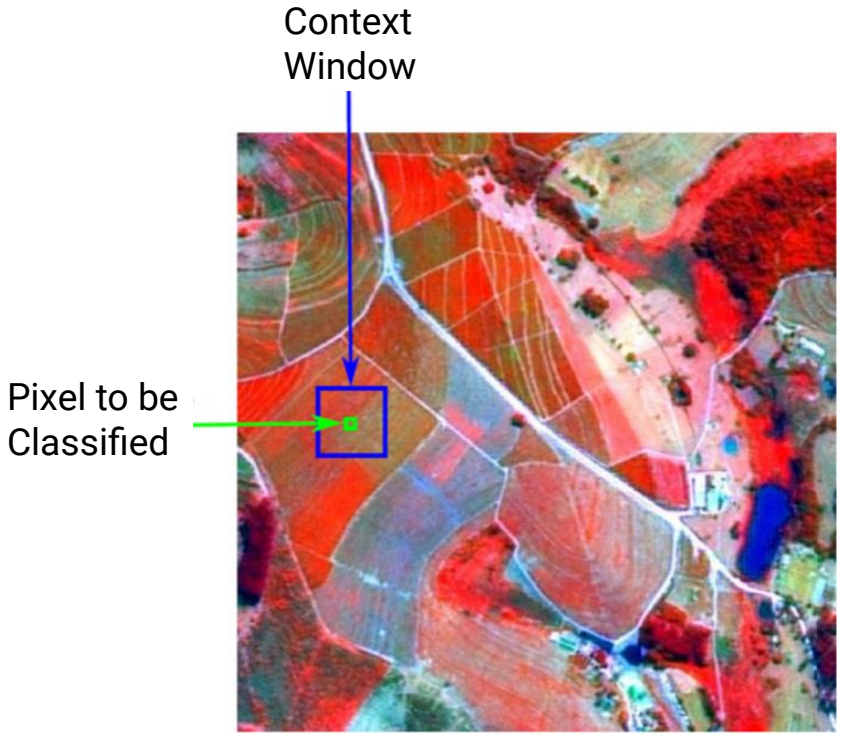




Deep Learning-based Semantic Segmentation

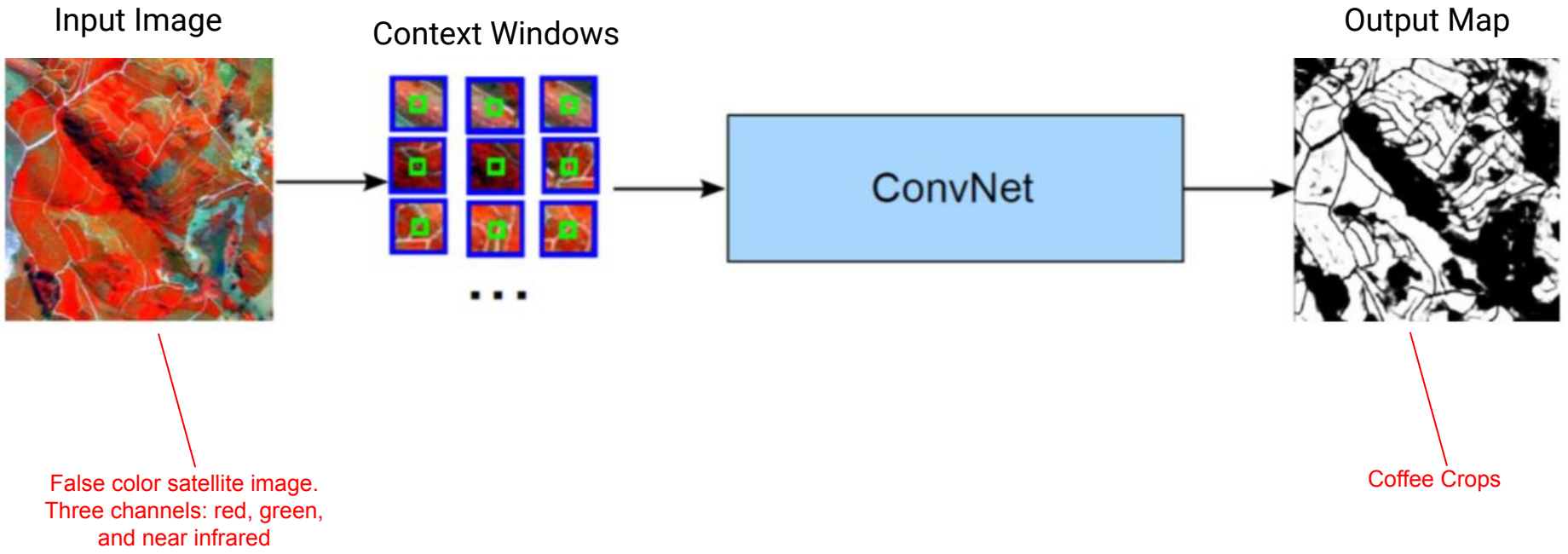
- Pixel-wise Classification
- Fully Convolutional Networks (FCNs)
- Encoder-Decoder Networks
  - DeconvNets
  - U-Nets
  - SegNets

Pixel-wise Classification

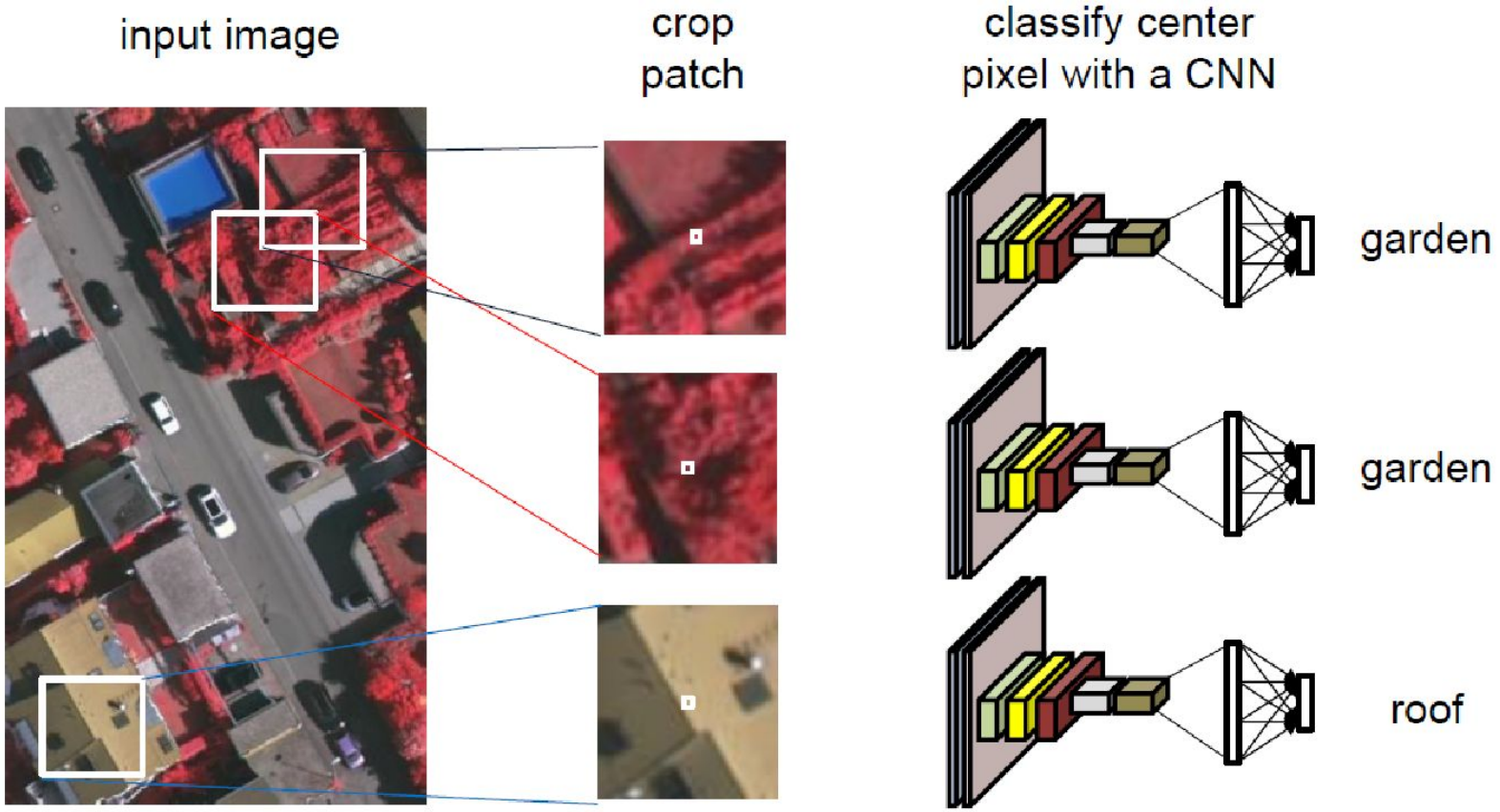


Nogueira et al. Learning to semantically segment high-resolution remote sensing images. ICPR 2016.

Pixel-wise Classification

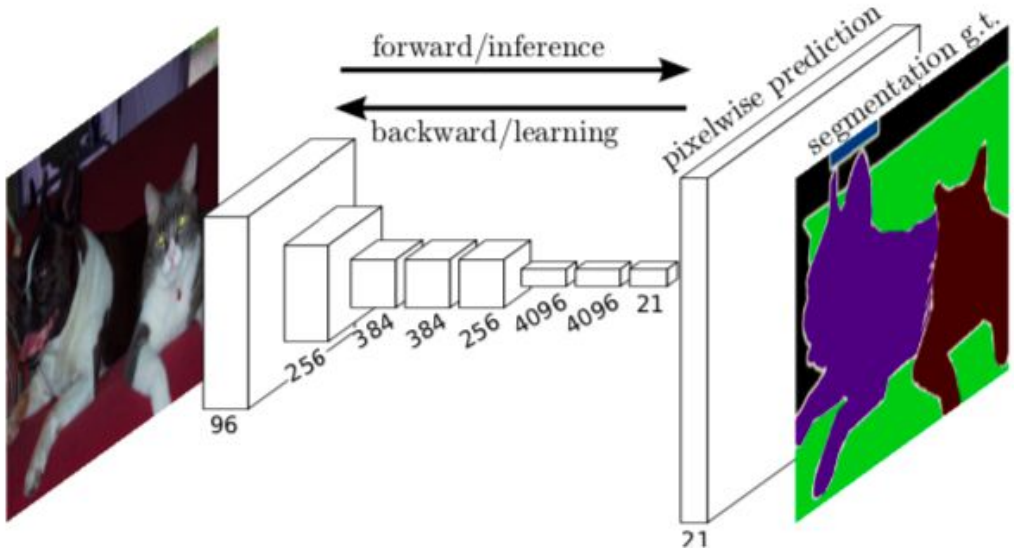


Nogueira et al. Learning to semantically segment high-resolution remote sensing images. ICPR 2016.



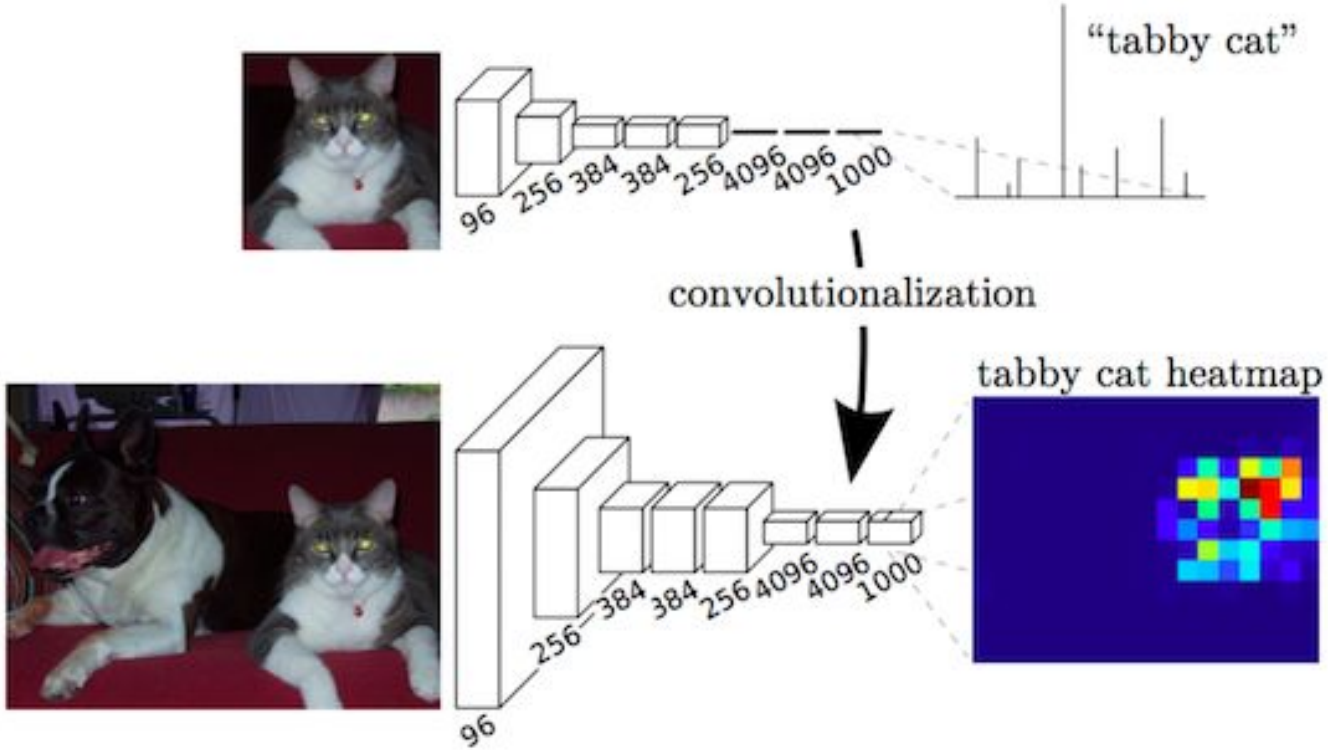
Problem: Redundant operations due to overlaps. Very inefficient!

Fully Convolutional Networks (FCNs)



J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. CVPR 2015.

Fully Convolutional Networks



J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. CVPR 2015.

Fully Convolutional Networks

- **Problem:** The last layer must have the same resolution as the image
- All the layers we've seen so far downsample the input, i.e. decrease the resolution
- How to upscale the last layer to return to the original size? How to recover the location that was lost during pooling operations?

  We will return to this question shortly.

- Note that the penultimate layer contains a feature map with a much lower resolution than the original image, and that it must have a number of channels/kernels equal to the number of classes in the dataset.

How to make upsampling in-network?

Given the following 2 x 2 input, how to obtain a 4 x 4 output?

Input:
2x2

1	2
3	4

How to make upsampling in-network?

Given the following 2 x 2 input, how to obtain a 4 x 4 output?

Input:
2x2

1	2
3	4

Output: 4x4

Nearest Neighbor

1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

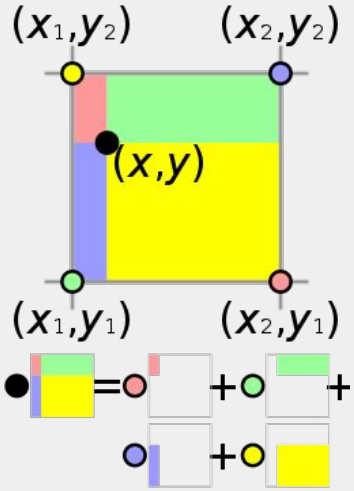
Bed of Nails
"Cama de pregos"

1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Bilinear Interpolation

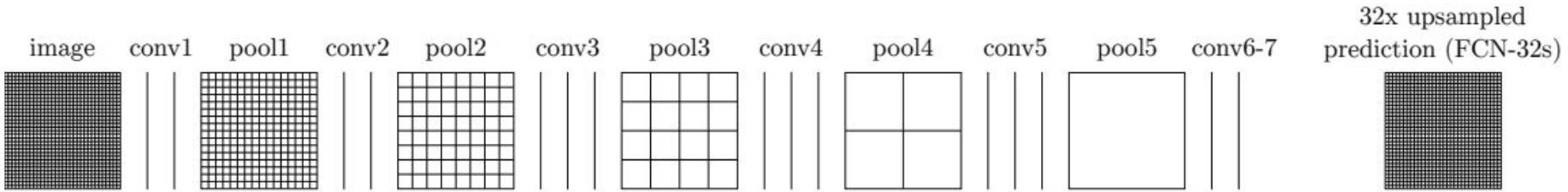
1		2	
3		4	

Bilinear interpolation.



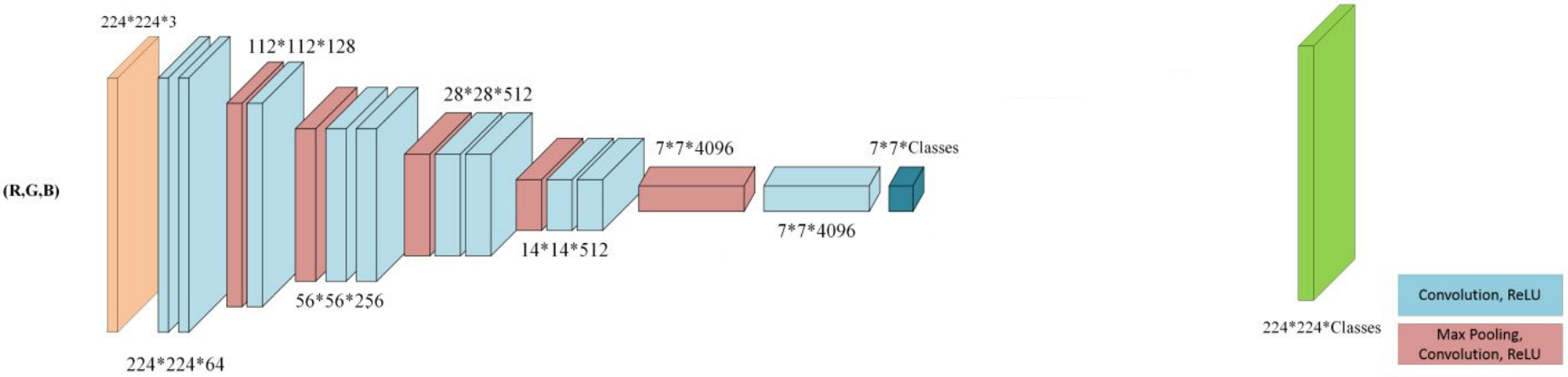
Compute each output y_{ij} from the 4 closest inputs using a linear map that depends only on the relative positions of the output and inputs

FCN-32s Architecture

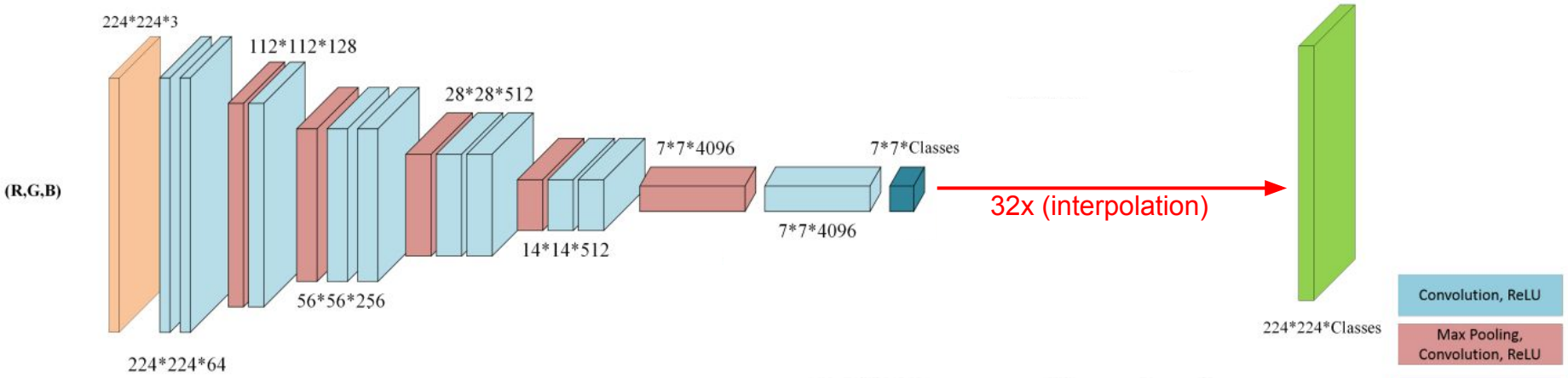


Long et al. Fully convolutional networks for semantic segmentation. In CVPR 2015.

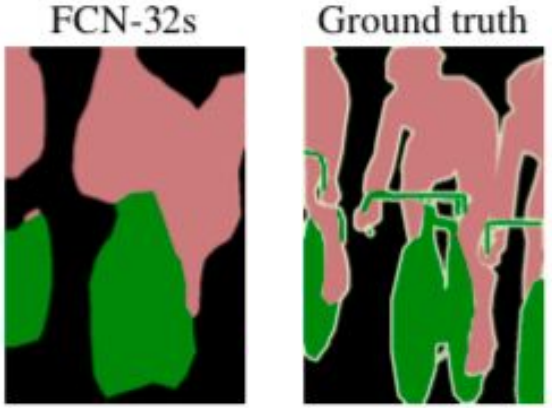
FCN-32s Architecture



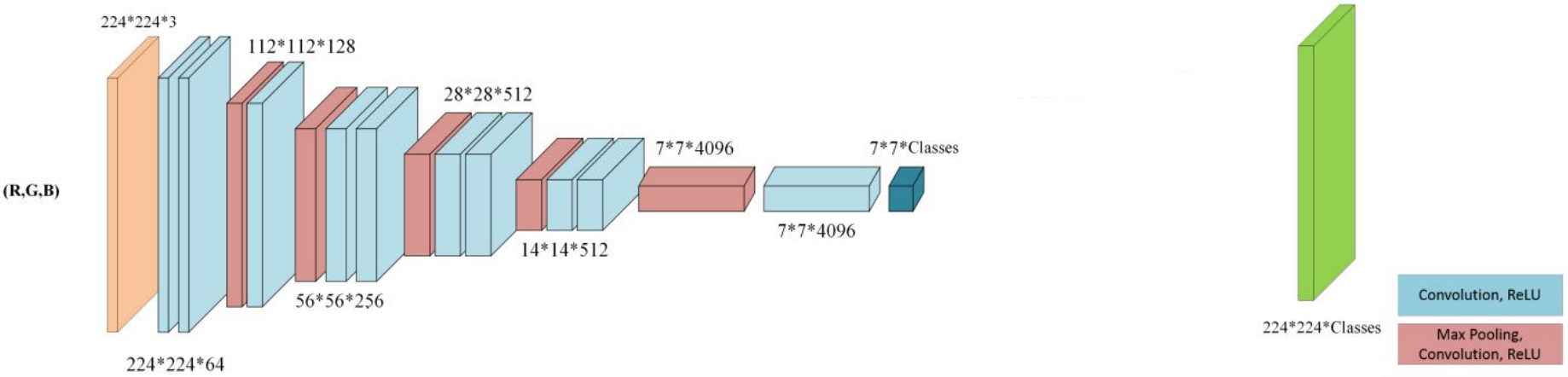
FCN-32s Architecture



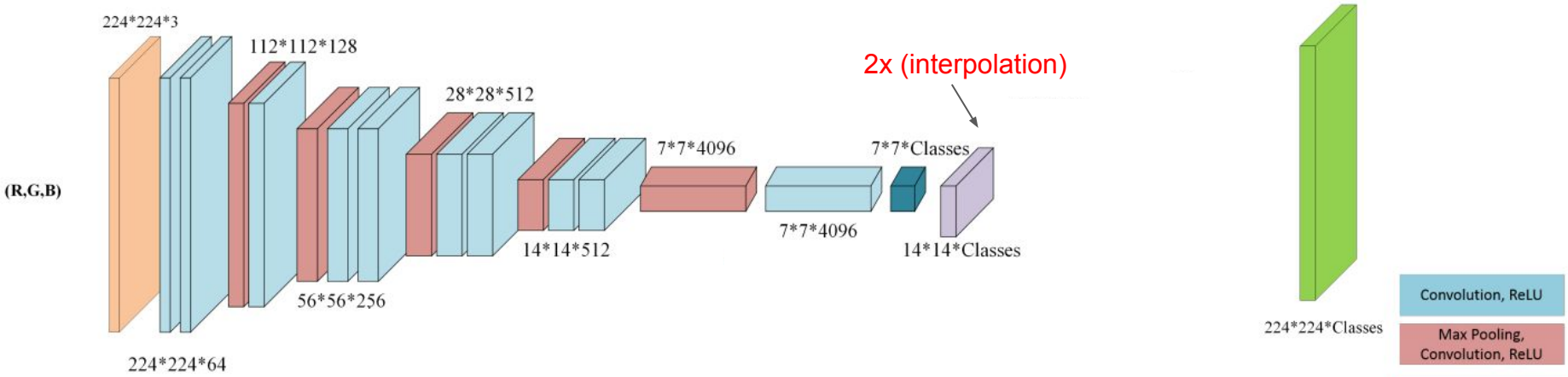
- FCN-32s: 32x conv7 (conv7 layer is upsampled 32x)
- Even after fine-tuning the pre-trained networks, the results are unsatisfactorily coarse



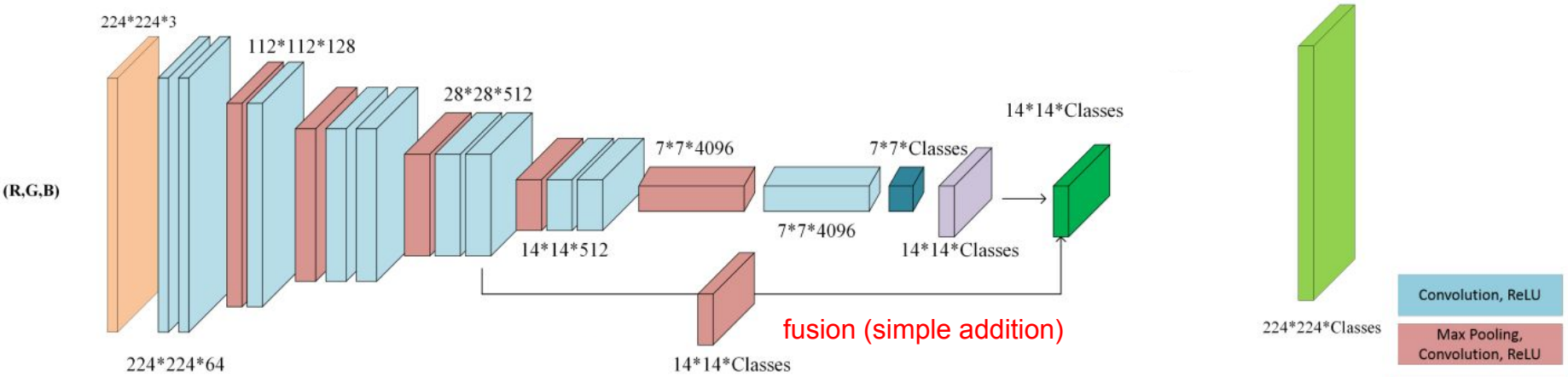
FCN-32s Architecture



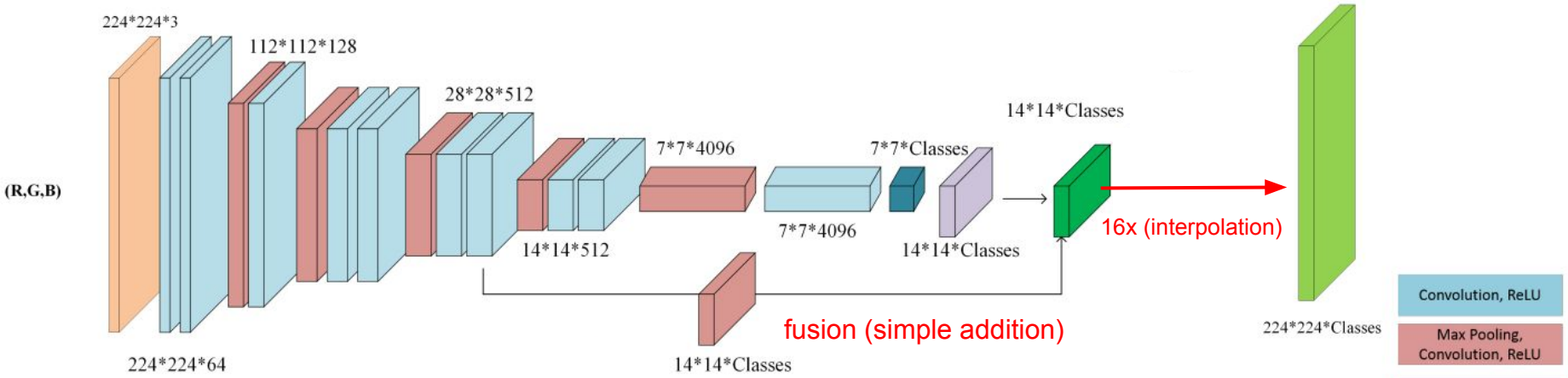
FCN-32s Architecture



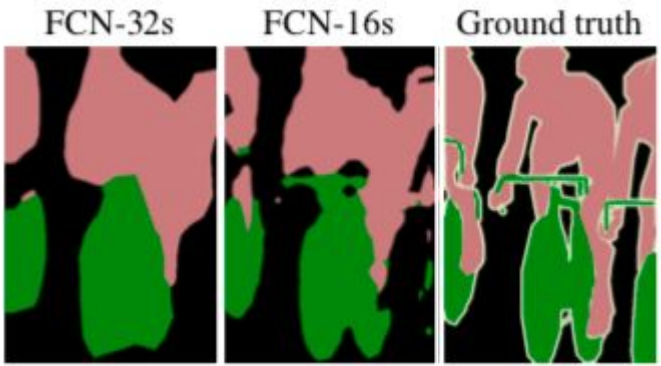
FCN-32s Architecture



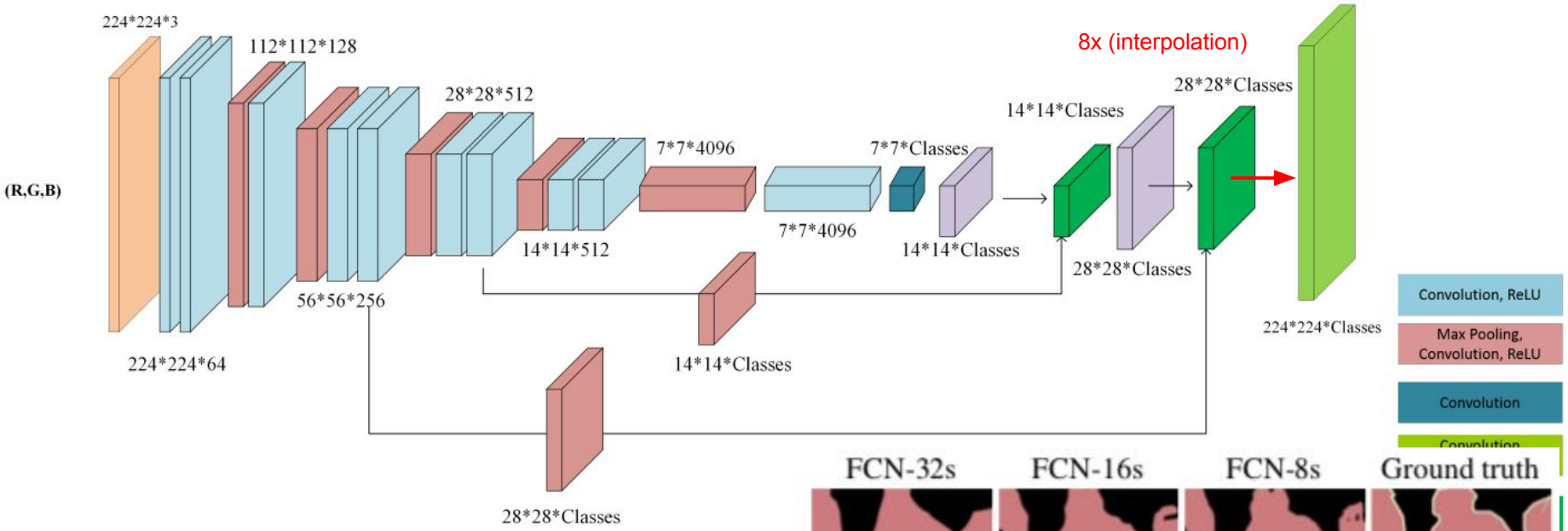
FCN-16s Architecture

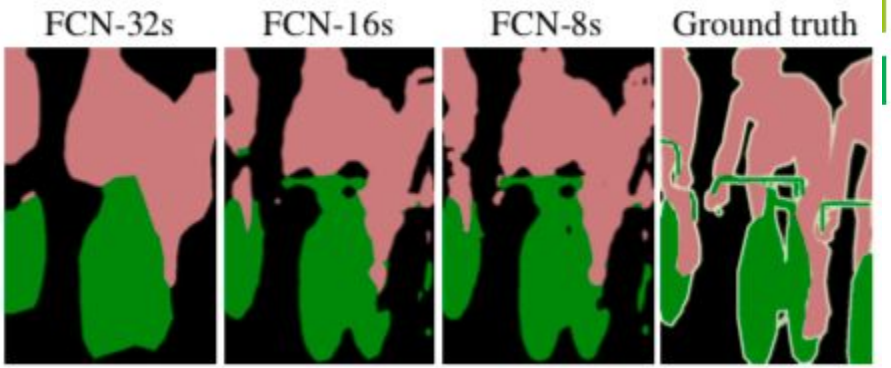


- Fusion of intermediate layers with prediction layers to complement spatial information of the final result
- FCN-16s: 2x conv7 + pool4; result is upsampled 16x
- Several advantages:
  - Fusion of high-level semantic information with low-level semantic information



FCN-8s Architecture





- FCN-8s: 4x conv7 + 2x pool4 + pool3; result is upsampled 8x
- To recover finer spatial information in the final decoder layers, I need to connect with shallower encoder layers (smaller receptive field)

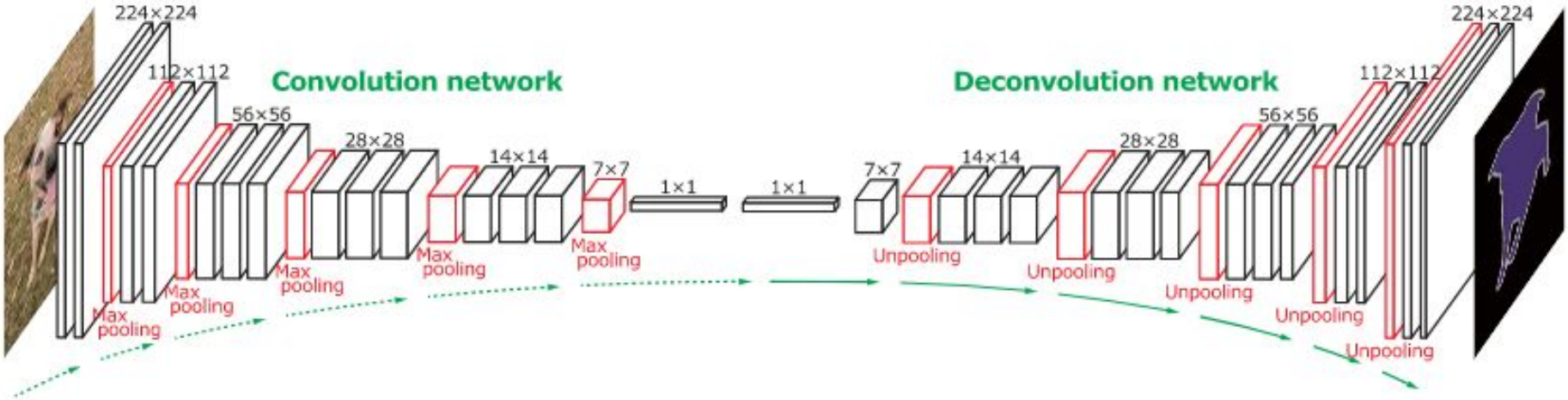
Main Contributions

- Popularized the use of end-to-end convolutional networks for semantic segmentation
- Adapted pre-trained networks with ImageNet for semantic segmentation
- Upsampling using interpolation Introduced “skip connections” to increase upsampling granularity

Semantic Segmentation

Deconvolution Networks
(DeconvNets)

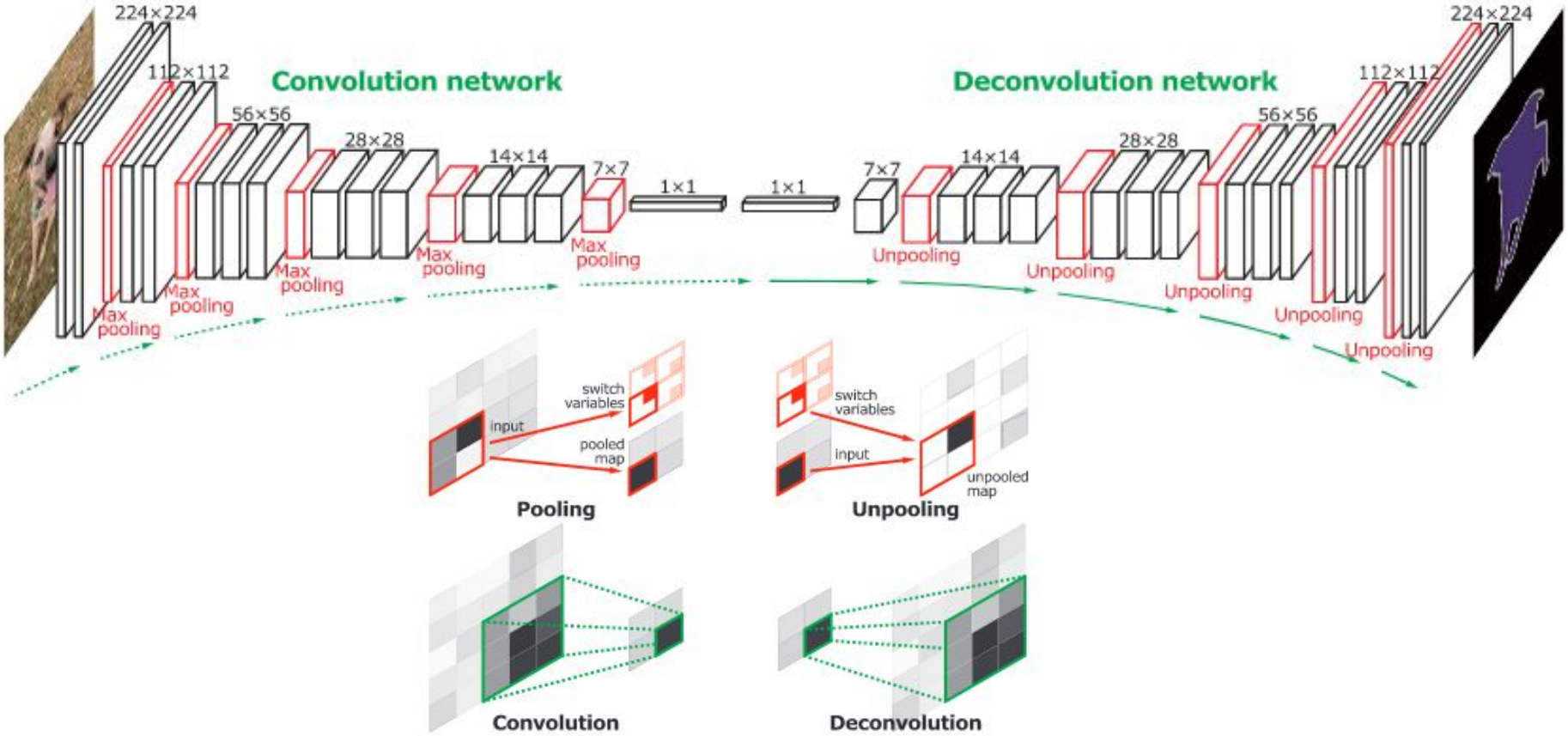
DeconvNets



"Mirrored" architecture!

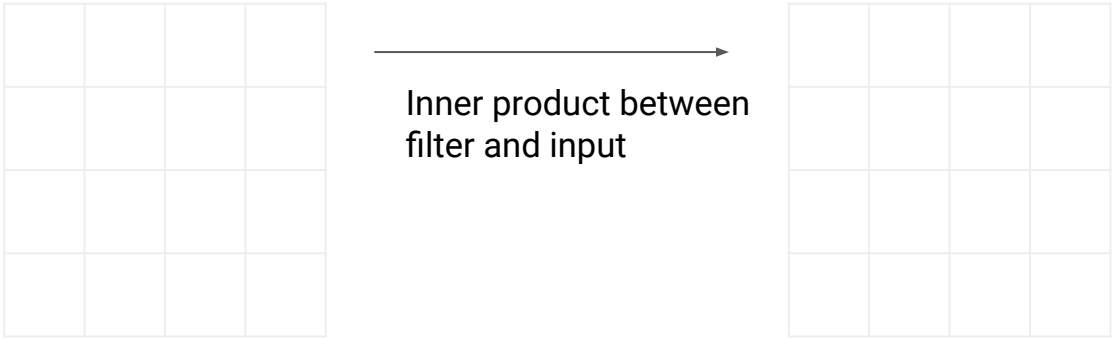
Noh, H., Hong, S., & Han, B. (2015). Learning deconvolution network for semantic segmentation. In Proceedings of the IEEE international conference on computer vision (pp. 1520-1528).

DeconvNets


224×224
112×112
56×56
28×28
14×14
7×7
1×1
1×1
7×7
14×14
28×28
56×56
112×112
224×224
Convolution network
Deconvolution network
Max pooling
Max pooling
Max pooling
Max pooling
Max pooling
Unpooling
Unpooling
Unpooling
Unpooling
Unpooling
switch variables
input
pooled map
Pooling
switch variables
input
unpooled map
Unpooling
Convolution
Deconvolution

Upsampling with learning: transpose convolution

As a reminder: typical 3 x 3 convolution, stride=1, pad=1

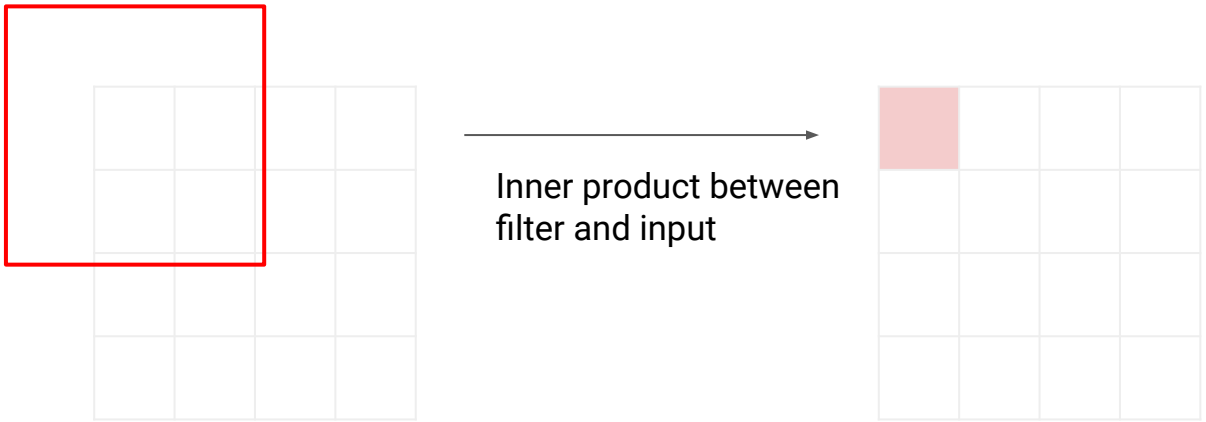


Input: 4x4

Output: 4x4

Upsampling with learning: transpose convolution

As a reminder: typical 3 x 3 convolution, stride=1, pad=1

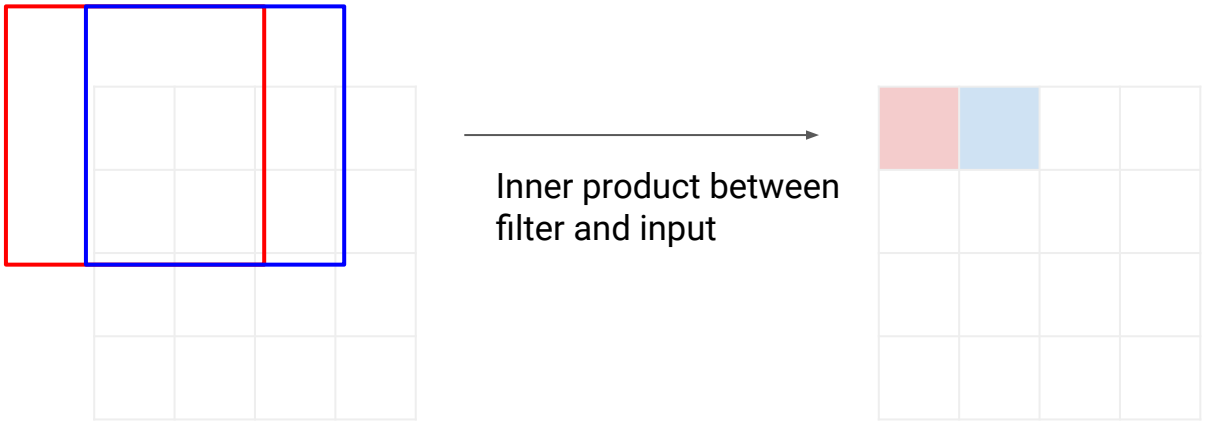


Input: 4x4

Output: 4x4

Upsampling with learning: transpose convolution

As a reminder: typical 3 x 3 convolution, stride=1, pad=1

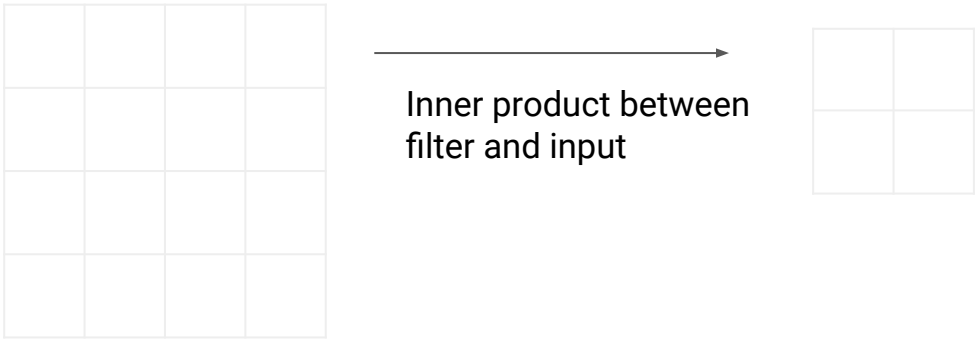


Input: 4x4

Output: 4x4

Upsampling with learning: transpose convolution

As a reminder: typical 3 x 3 convolution, stride=2, pad=1



Input: 4x4

Output: 2x2

Upsampling with learning: transpose convolution

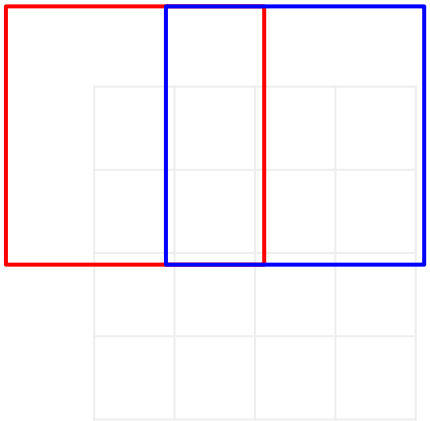
As a reminder: typical 3 x 3 convolution, stride=2, pad=1

Input: 4x4

Output: 2x2

Upsampling with learning: transpose convolution

As a reminder: typical 3 x 3 convolution, stride=2, pad=1



Inner product between filter and input

Input: 4x4

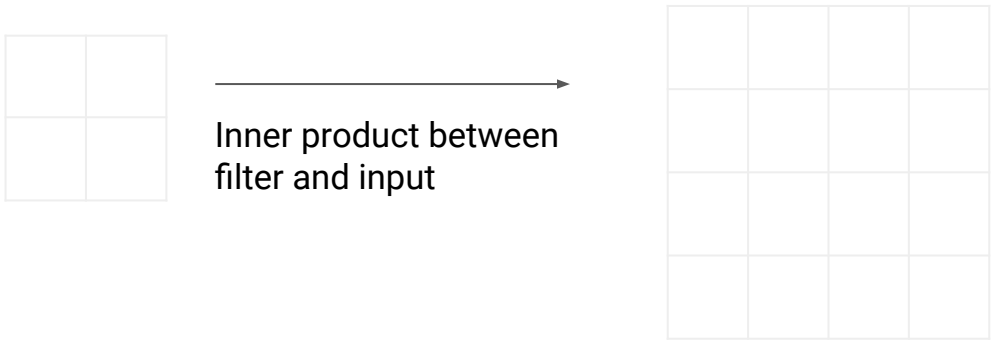
Output: 2x2

Filter moves 2 pixels in the input for each pixel in the output

Stride gives the ratio between the displacement at the entrance and exit

Upsampling with learning: transpose convolution

transpose convolution 3 x 3, stride=2, pad=1

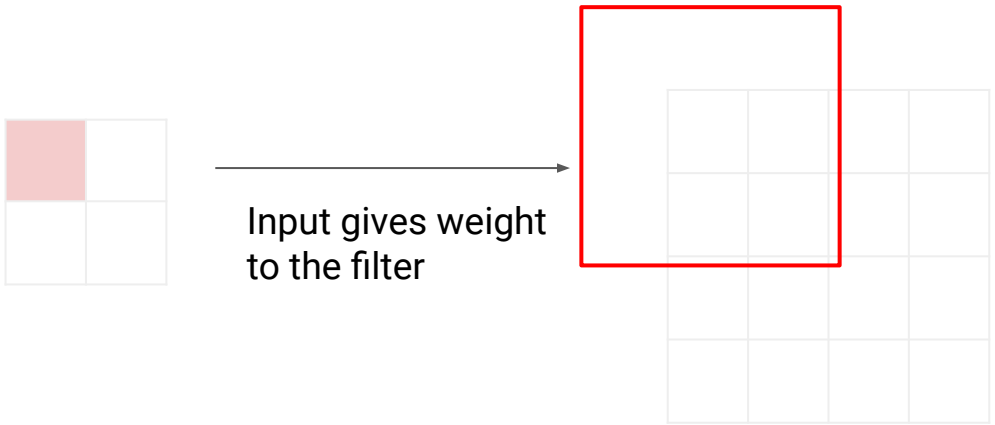


Input: 2x2

Output: 4x4

Upsampling with learning: transpose convolution

transpose convolution 3 x 3, stride=2, pad=1

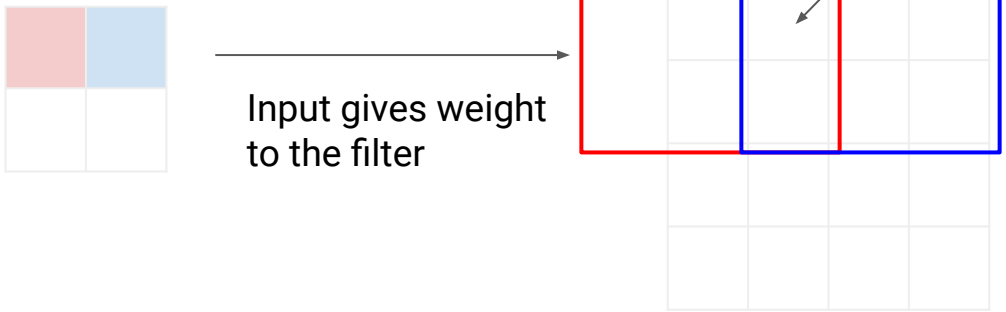


Input: 2x2

Output: 4x4

Upsampling with learning: transpose convolution

transpose convolution 3 x 3, stride=2, pad=1



We add where output overlaps

Input gives weight to the filter

Filter moves 2 pixels in the output for each pixel in the input

Stride gives the ratio between the movement at the exit and entry

Input: 2x2

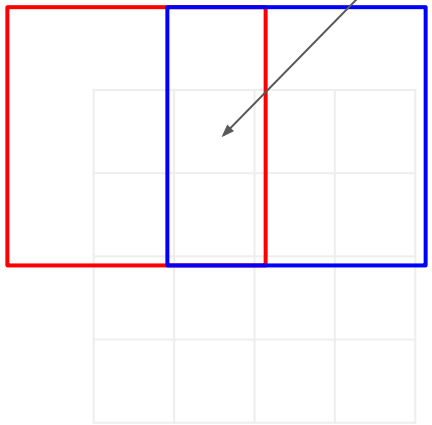
Output: 4x4

Upsampling with learning: transpose convolution

transpose convolution 3 x 3, stride=2, pad=1

Other names:
- Deconvolution 👎
- Upconvolution
- Fractionally strided convolution
- Backward strided convolution

Input gives weight to the filter



We add where output overlaps

Filter moves 2 pixels in the output for each pixel in the input

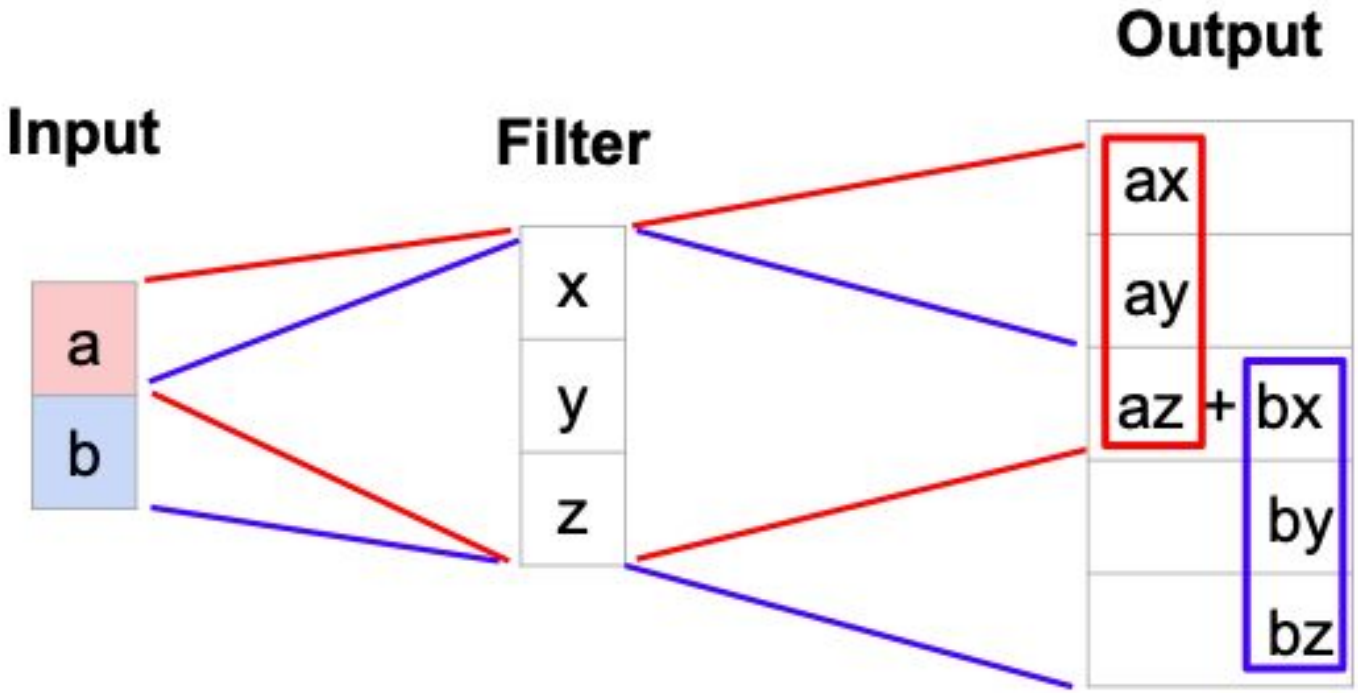
Stride gives the ratio between the movement at the exit and entry

Input: 2x2

Output: 4x4

Transpose convolution: 1D example

Example with stride=2



Output contains copies of the filter weighted by the input, summing where output overlap occurs

The floor of o/i is 2, o: output size i: input size

Convolution as matrix multiplication (Example 1D)

Let **x** = (x,y,z) be the weight vector and a = (a,b,c,d) be the input vector

We can express convolution in terms of matrix multiplication

$$\mathbf{x} * \mathbf{a} = \mathbf{X}\mathbf{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & x & y & z & 0 & 0 \\ 0 & 0 & x & y & z & 0 \\ 0 & 0 & 0 & x & y & z \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ ax + by + cz \\ bx + cy + dz \\ cx + dy \end{bmatrix}$$

Example: 1D Conv, size 3, stride=1, padding=1

Convolution as matrix multiplication (Example 1D)

Let **x** = (x,y,z) be the weight vector and a = (a,b,c,d) be the input vector

We can express convolution in terms of matrix multiplication

$$\mathbf{x} * \mathbf{a} = \mathbf{X}\mathbf{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & x & y & z & 0 & 0 \\ 0 & 0 & x & y & z & 0 \\ 0 & 0 & 0 & x & y & z \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ ax + by + cz \\ bx + cy + dz \\ cx + dy \end{bmatrix}$$

Example: 1D Conv, size 3, stride=1, padding=1

Transposed convolution multiplies by the transpose of the same matrix

$$\mathbf{x} *^{\top} \mathbf{a} = \mathbf{X}^{\top}\mathbf{a}$$

$$\begin{bmatrix} x & 0 & 0 & 0 \\ y & x & 0 & 0 \\ z & y & x & 0 \\ 0 & z & y & x \\ 0 & 0 & z & y \\ 0 & 0 & 0 & z \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} ax \\ ay + bx \\ az + by + cx \\ bz + cy + dx \\ cz + dy \\ dz \end{bmatrix}$$

When stride=1, transpose convolution works like a regular convolution (with different padding rules)

Convolution as matrix multiplication (Example 1D)

Let **x** = (x,y,z) be the weight vector and a = (a,b,c,d) be the input vector

We can express convolution in terms of matrix multiplication

$$\mathbf{x} * \mathbf{a} = \mathbf{X}\mathbf{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & x & y & z & 0 & 0 \\ 0 & 0 & x & y & z & 0 \\ 0 & 0 & 0 & x & y & z \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ ax + by + cz \\ bx + cy + dz \\ cx + dy \end{bmatrix}$$

Example: 1D Conv, size 3, stride=2, padding=1

Transposed convolution multiplies by the transpose of the same matrix

$$\mathbf{x} *^{\top} \mathbf{a} = \mathbf{X}^{\top}\mathbf{a}$$

$$\begin{bmatrix} x & 0 & 0 & 0 \\ y & x & 0 & 0 \\ z & y & x & 0 \\ 0 & z & y & x \\ 0 & 0 & z & y \\ 0 & 0 & 0 & z \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} ax \\ ay + bx \\ az + by + cx \\ bz + cy + dx \\ cz + dy \\ dz \end{bmatrix}$$

When stride >1, transposed convolution is no longer a normal convolution

Convolution as matrix multiplication (Example 1D)

Let **x** = (x,y,z) be the weight vector and a = (a,b,c,d) be the input vector

We can express convolution in terms of matrix multiplication

$$\mathbf{x} * \mathbf{a} = \mathbf{X}\mathbf{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & 0 & x & y & z & 0 \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ bx + cy + dz \end{bmatrix}$$

Exemplo: 1D Conv, tamanho 3, stride=2, padding=1

Convolution as matrix multiplication (Example 1D)

Let **x** = (x,y,z) be the weight vector and a = (a,b,c,d) be the input vector

We can express convolution in terms of matrix multiplication

$$\mathbf{x} * \mathbf{a} = \mathbf{X}\mathbf{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & 0 & x & y & z & 0 \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ bx + cy + dz \end{bmatrix}$$

Exemplo: 1D Conv, tamanho 3, stride=2, padding=1

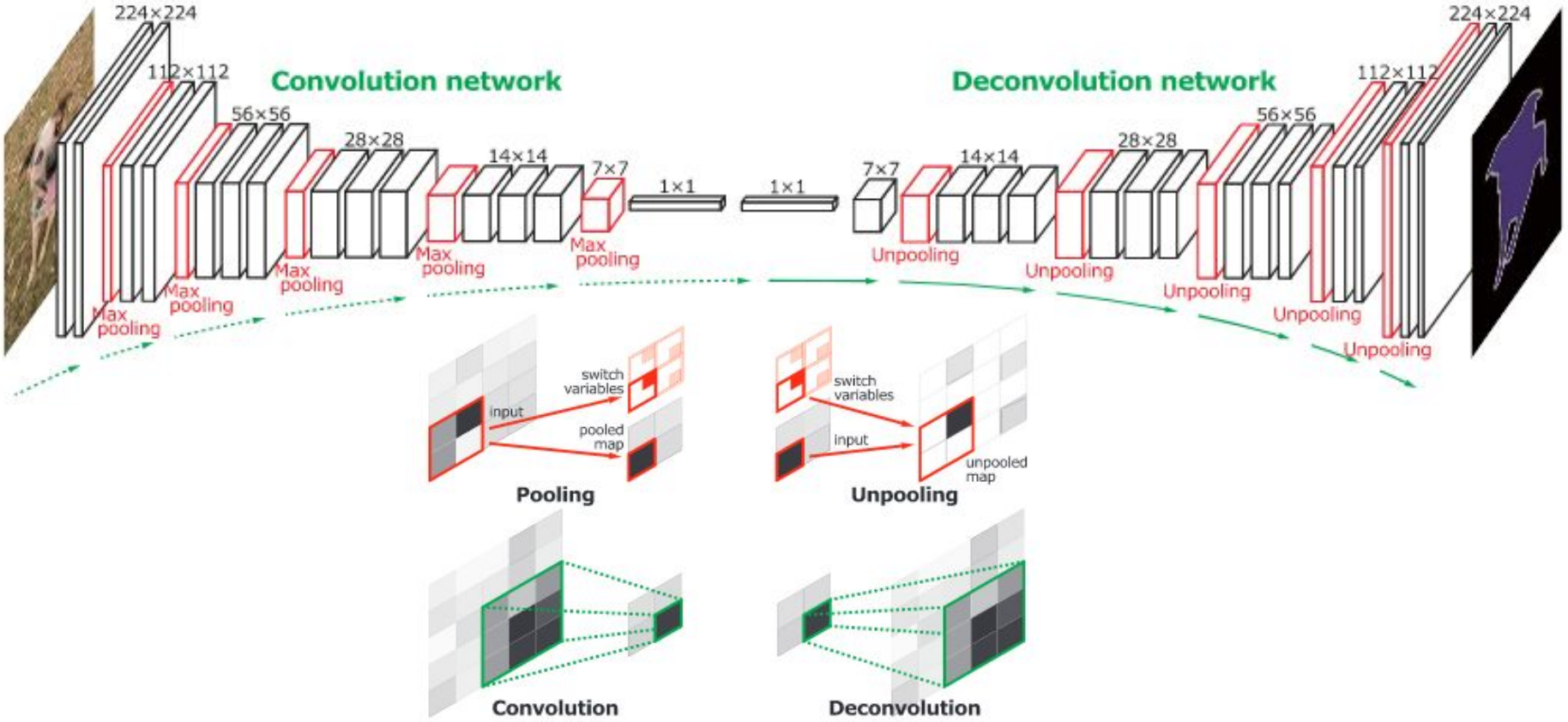
Transposed convolution multiplies by the transpose of the same matrix

$$\mathbf{x} *^{\top} \mathbf{a} = \mathbf{X}^{\top}\mathbf{a}$$

$$\begin{bmatrix} x & 0 \\ y & 0 \\ z & x \\ 0 & y \\ 0 & z \\ 0 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} ax \\ ay \\ az + bx \\ by \\ bz \\ 0 \end{bmatrix}$$

Quando stride >1, convolução transposta não é mais uma convolução normal

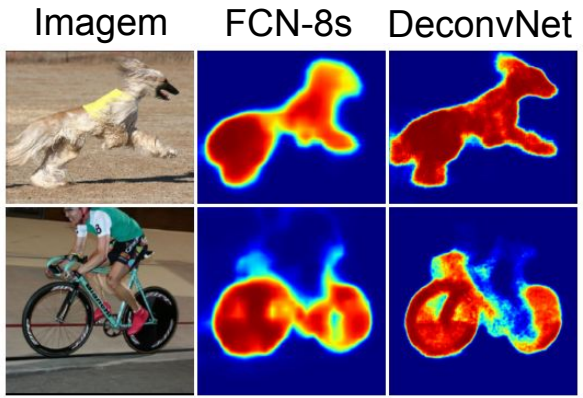
DeconvNets

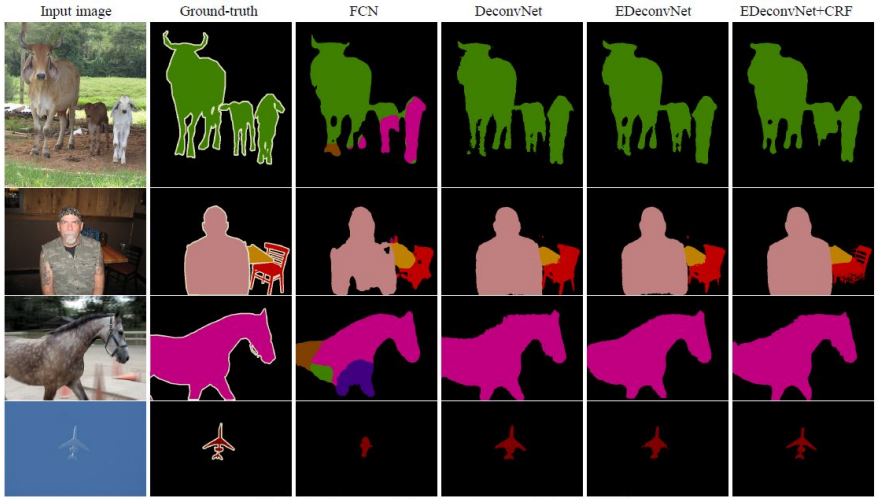

224×224
112×112
56×56
28×28
14×14
7×7
1×1
1×1
7×7
14×14
28×28
56×56
112×112
224×224
Convolution network
Deconvolution network
Max pooling
Max pooling
Max pooling
Max pooling
Max pooling
Unpooling
Unpooling
Unpooling
Unpooling
Unpooling
switch variables
input
pooled map
Pooling
switch variables
input
unpooled map
Unpooling
Convolution
Deconvolution

DeconvNets

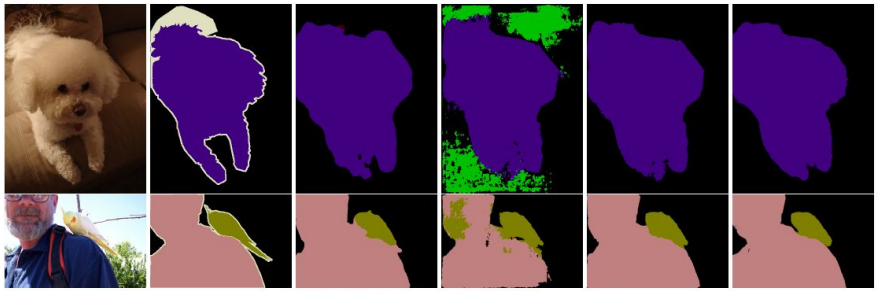
Problems:

- Hard to converge
- Too many parameters

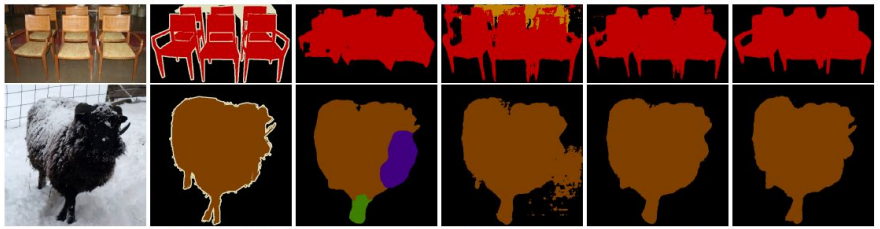




(a) Examples that our method produces better results than FCN [17].



(b) Examples that FCN produces better results than our method.



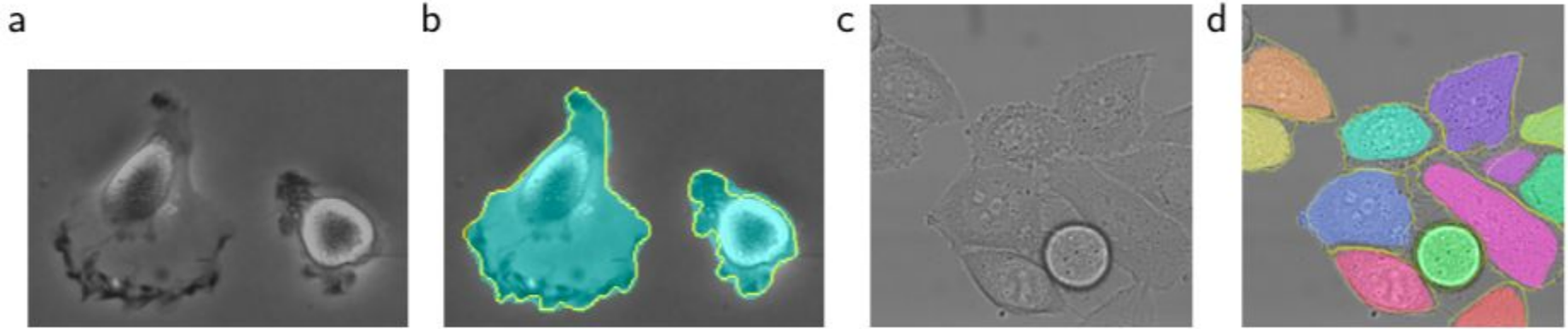
(c) Examples that inaccurate predictions from our method and FCN are improved by ensemble.

Semantic Segmentation

U-Nets

U-Net

- Proposed in the context of biomedical image processing
  - ISBI challenge for segmentation of neuronal structures
  - Need to perform dense classification (per pixel)
  - Few training examples (~30 per application)
  - Cells that touch each other; very small space between objects
- Reusable solution across different domains



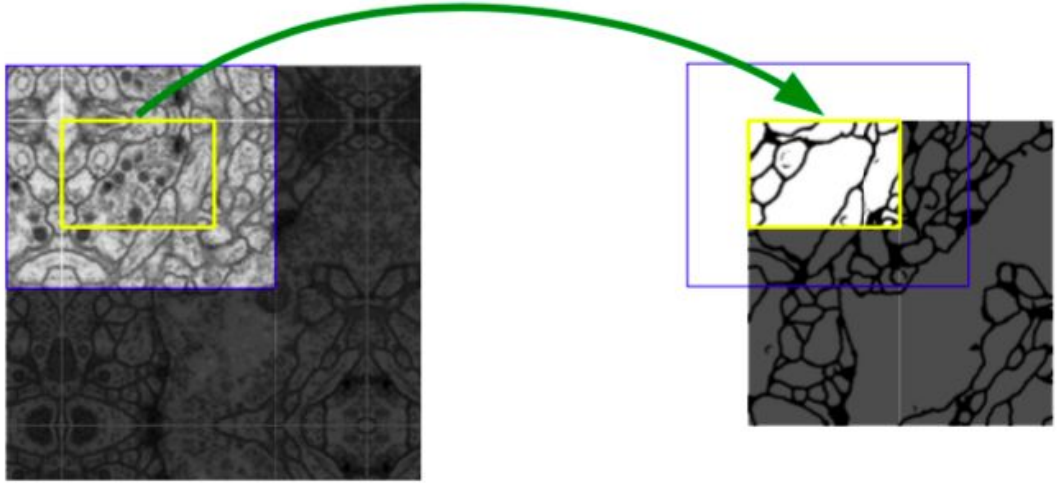
PhC-U373: 35 images

PhC-U373: 20 images

Ronneberger et al. U-net: Convolutional networks for biomedical image segmentation. MICCAI 2015.

U-Net: main ideas

- Encoder-decoder architecture
- Image Augmentation
- Padding: instead of completing with 0's, mirror the image on the edges. Technique used in the image that enters the network.



U-Net: main ideas

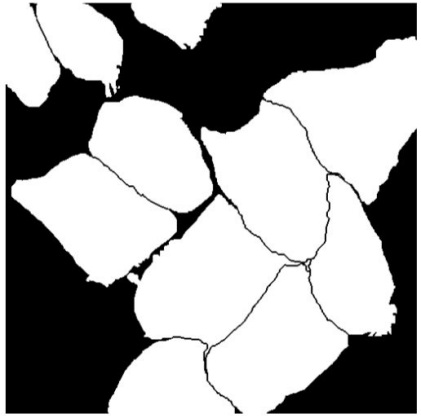
- Loss function that gives weights to different pixels
  - Balancing the different frequencies of the classes (in the images there are more examples of one class than others)
  - Force the network to learn small separation edges present between touching cells

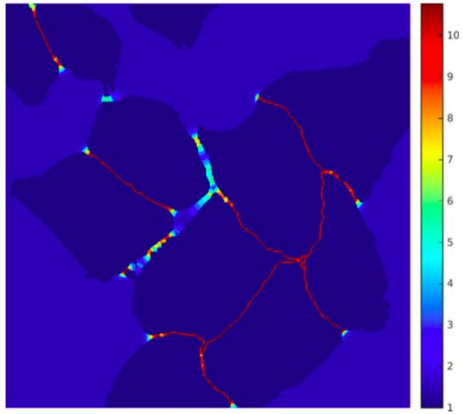
$$w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \cdot \exp\left(-\frac{(d_1(\mathbf{x}) + d_2(\mathbf{x}))^2}{2\sigma^2}\right)$$

Class frequency compensation

Distance to nearest edge

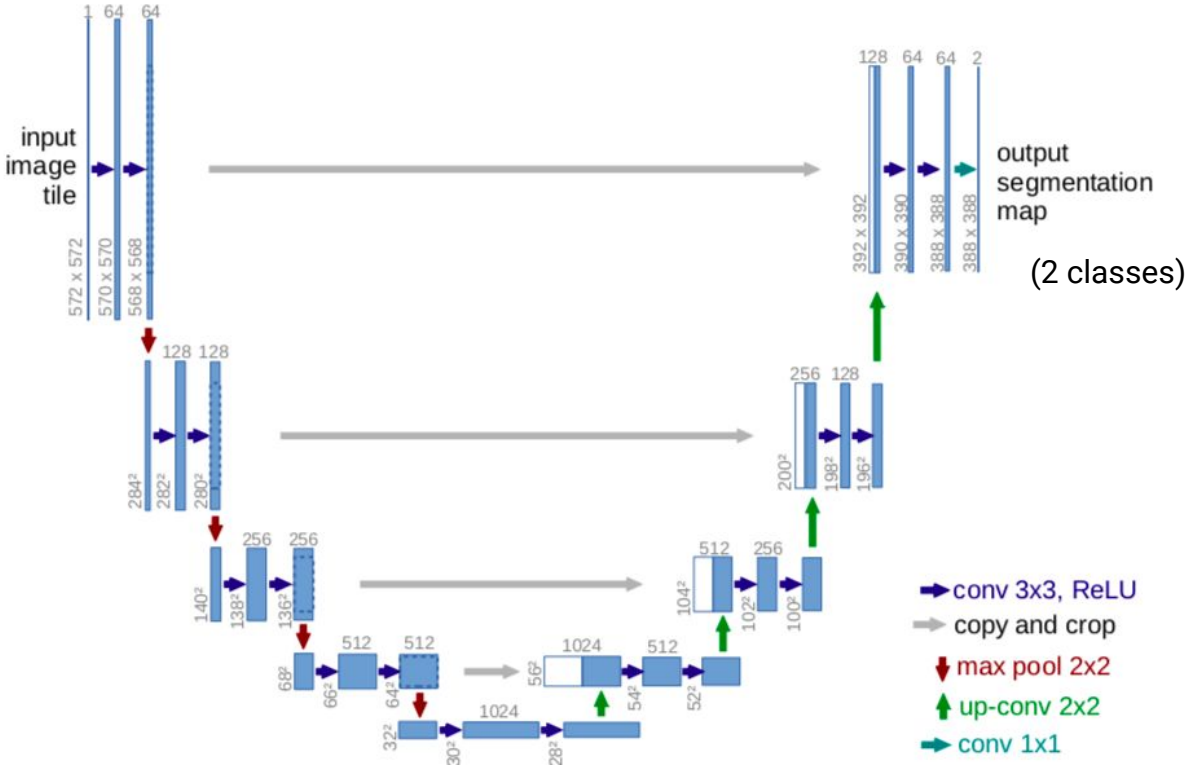
Distance up to 2nd. nearest edge

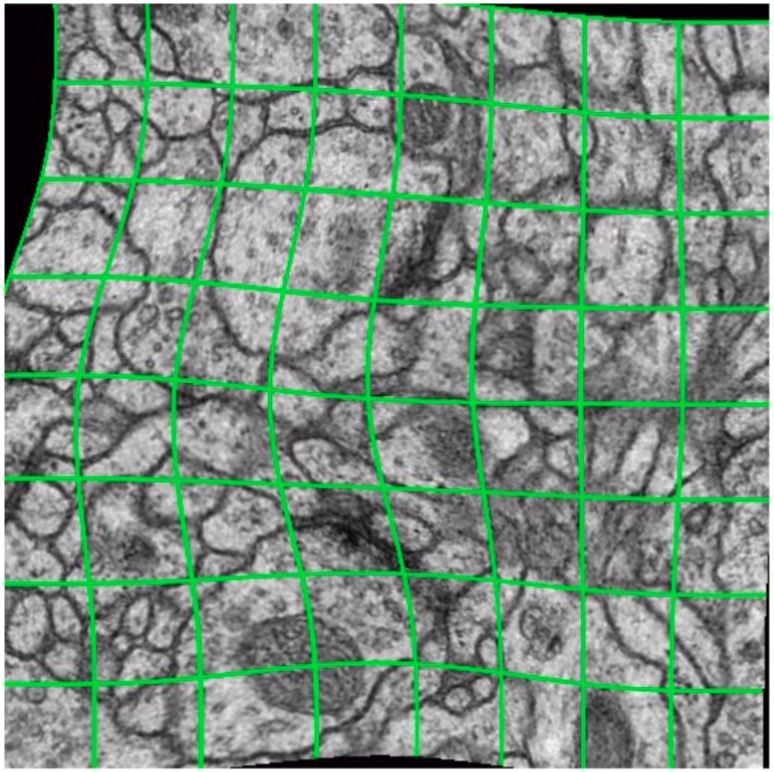




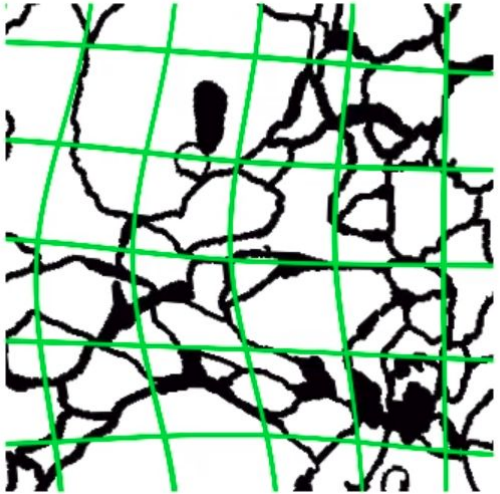
U-Net

- Layers dedicated to spatial information retrieval
- Decoder: concatenate feature maps from corresponding downsampling layers to obtain more accurate locations





resulting deformed image
(for visualization: no rotation, no shift, no extrapolation)



correspondingly deformed
manual labels

U-Net main contributions

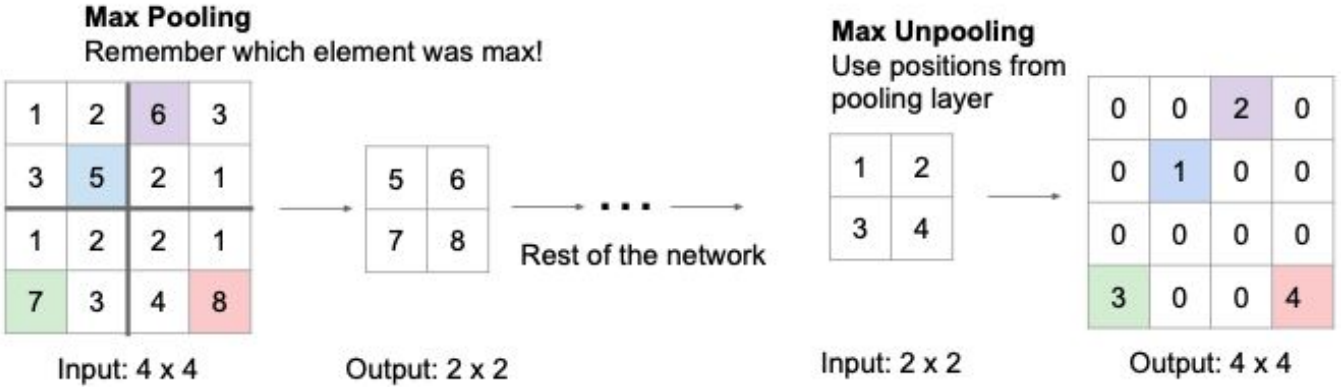
- Layers dedicated to spatial information retrieval
- In the decoder: concatenate feature maps from corresponding downsampling layers to obtain more accurate locations
- Data Augmentation for Microscopic Imaging
- Cost function that allows you to learn small separation edges between objects

Semantic Segmentation

SegNet

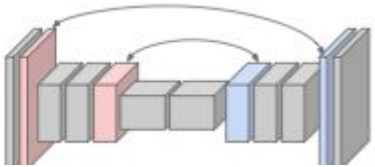
How to do in-network upsampling (unpooling)?

When we do max pooling, we lose information about which layer element was used. We can save these positions and use them later:



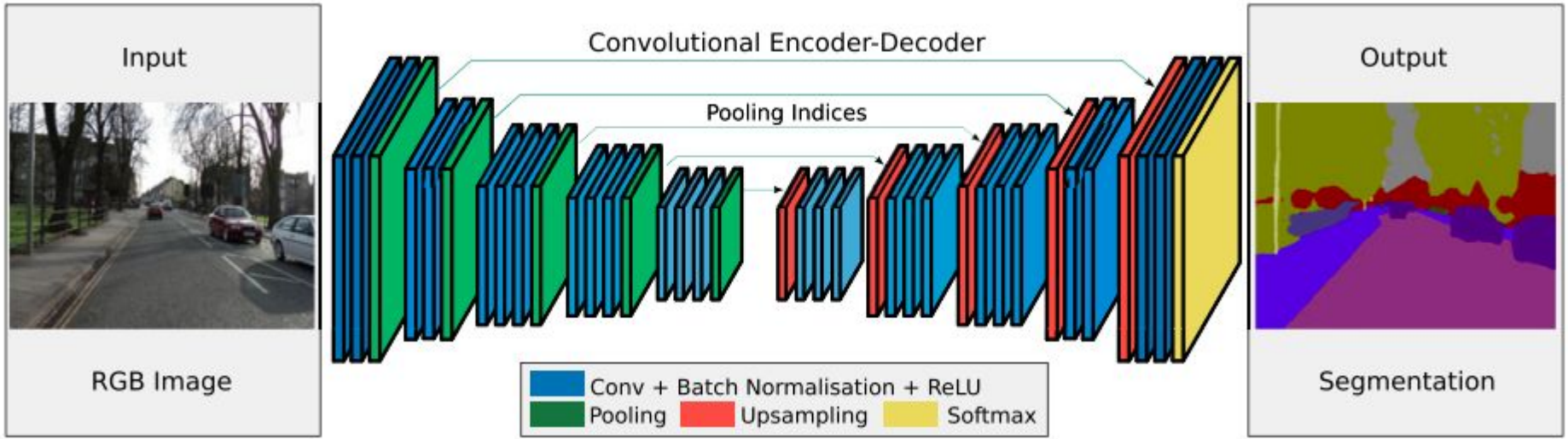
Similar to bed of nails, but places corresponding elements in their original positions

Corresponding pairs of downsampling and upsampling layers



SegNet

- Using Encoder pooling indexes for Decoder rebuild



Badrinarayanan et al. SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation." IEEE TPAMI 2016.

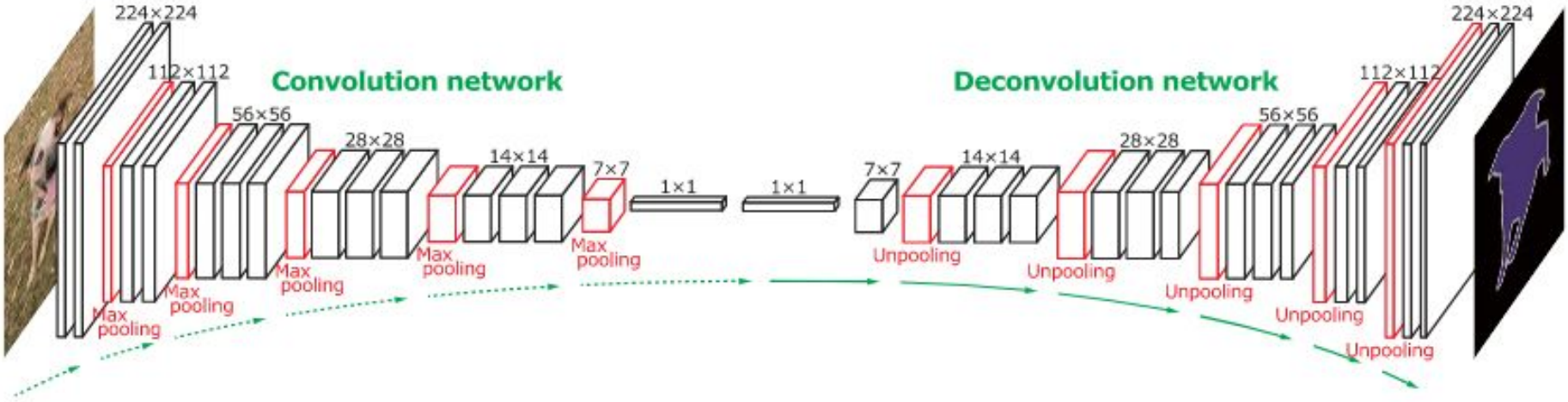
Semantic Segmentation

Dilated Convolution
(a.k.a. *Atrous Convolution*)

Segmentation with CNNs

Problem:

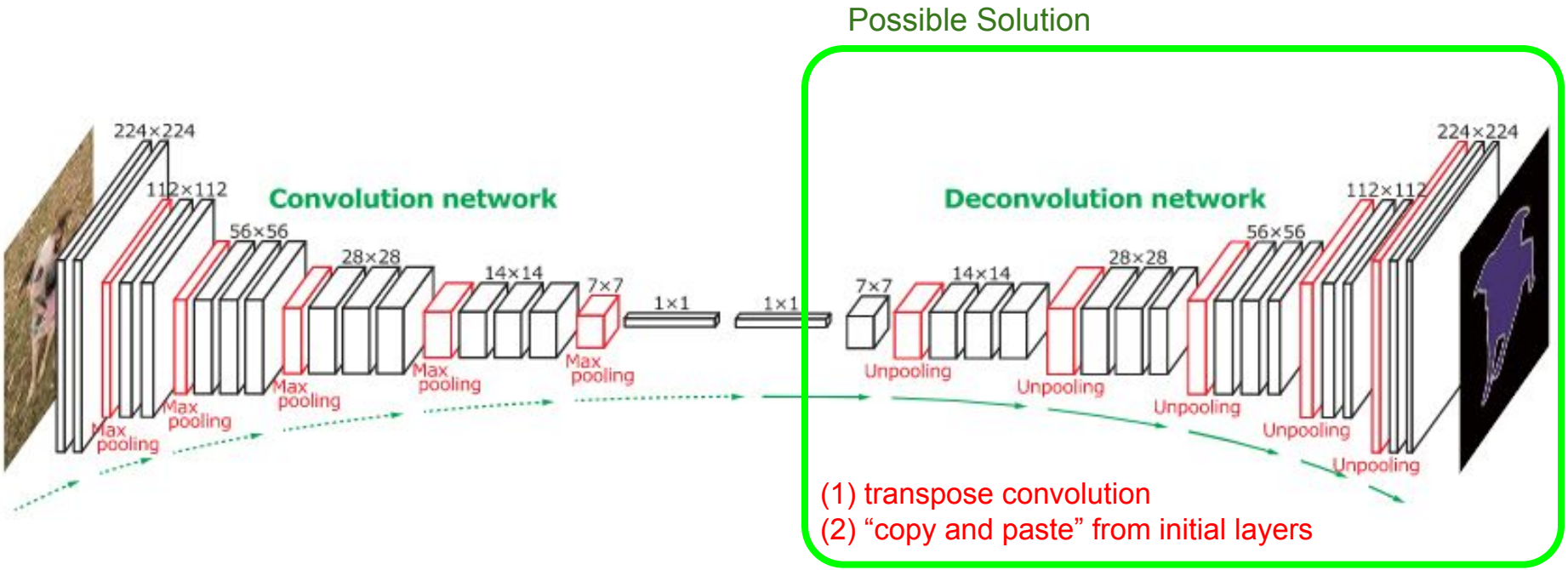
- **Down-sampling** causes loss of spatial information.



Segmentation with CNNs

Problem:

- **Down-sampling** causes loss of spatial information.



Segmentation with CNNs

Problem:

- x **Down-sampling** causes loss of spatial information**.**

Alternative solution:

- Dilated Convolution (‘Holes’ algorithm).

Dilated Convolution

Introduces a parameter for the convolutional layers: the dilation rate.

- Dilated convolution for the 1-D signal:

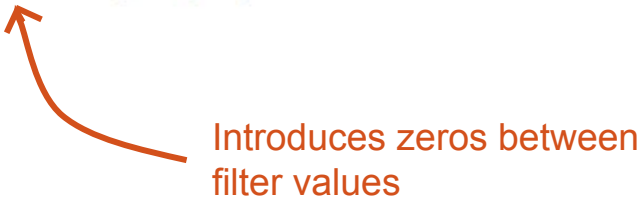
$$y[i] = \sum_{k=1}^{K} x[i + r \cdot k] w[k]$$

$x[i]$ 1-D input signal

$w[k]$ filter of length K

r *rate* parameter corresponds to the stride with which we sample the input signal.

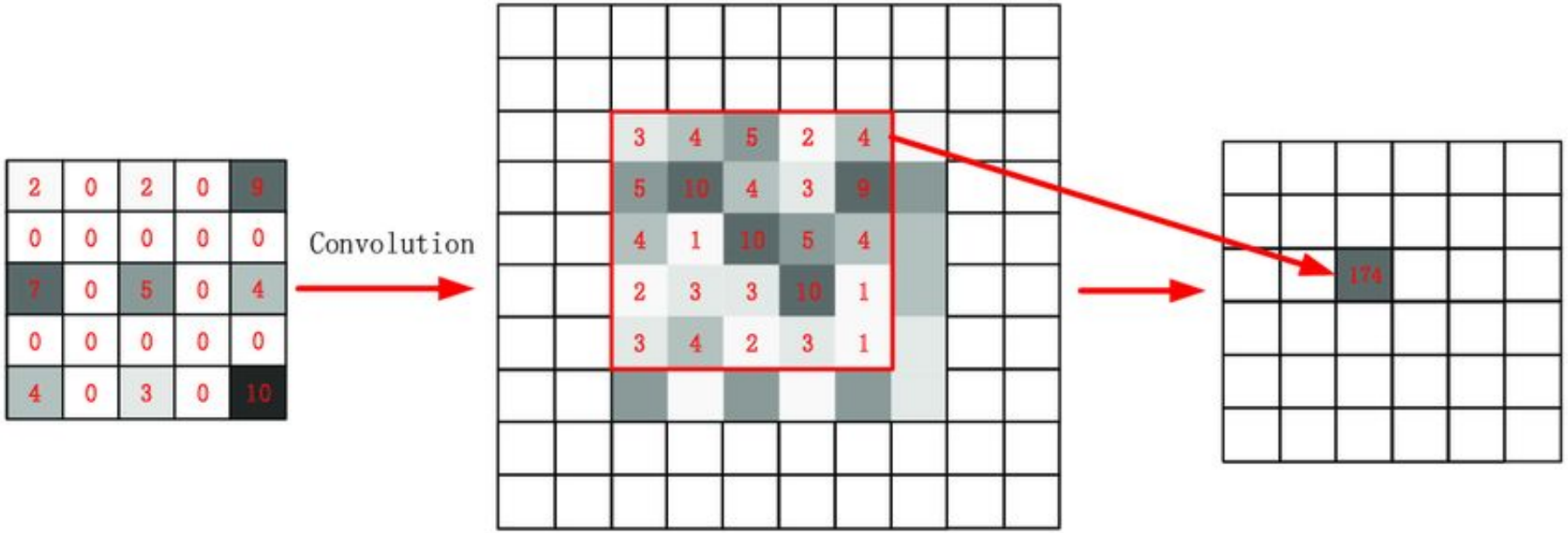
$y[i]$ output of atrous convolution.



Note: standard convolution is a special case where the rate is r = 1.

Dilated Convolution

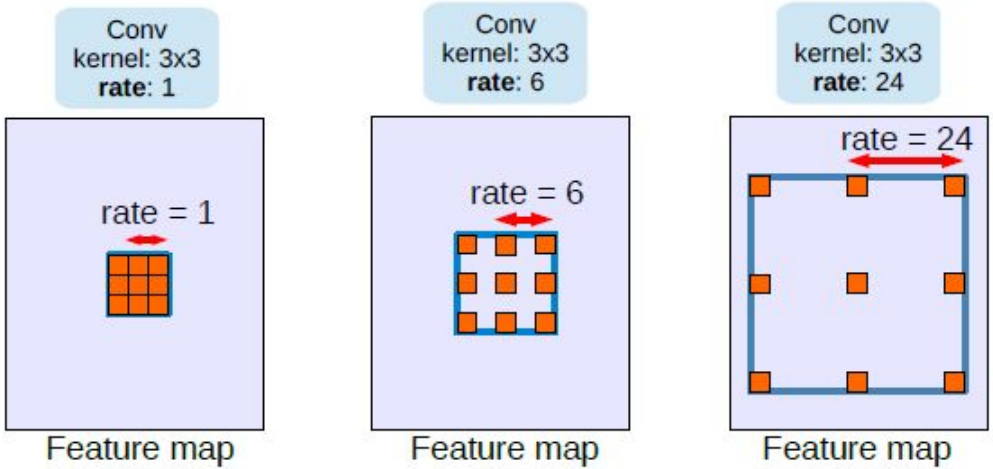
Introduces a parameter for the convolutional layers: the dilation rate.



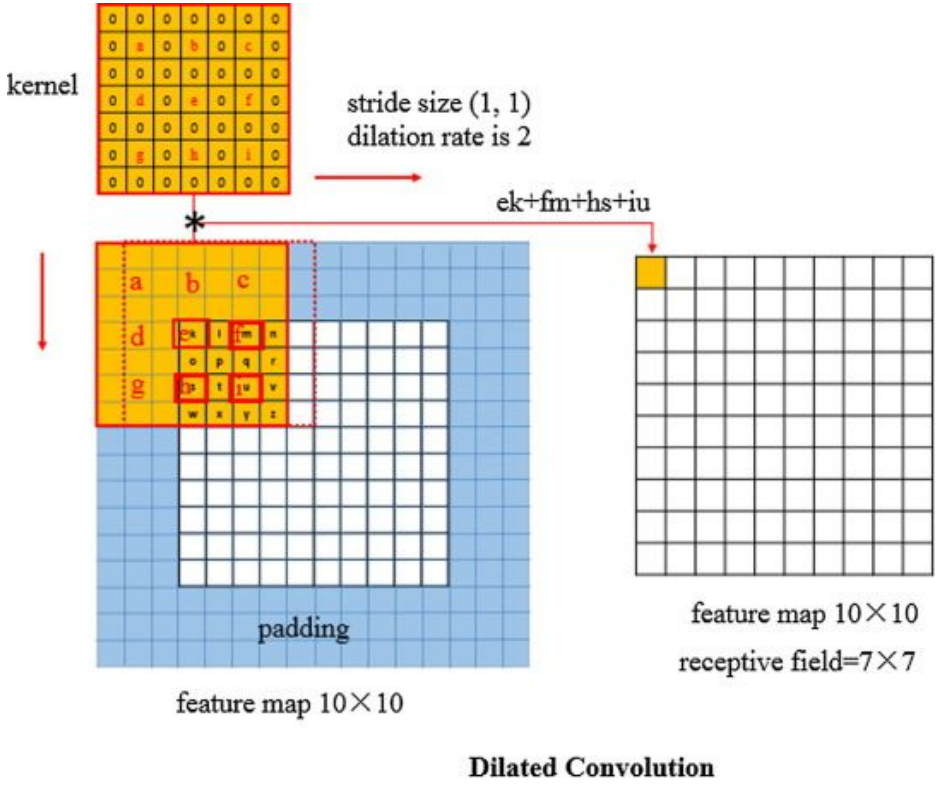
Dilated Convolution

Introduces a parameter for the convolutional layers: the dilation rate.

- Effect:
  - A 3x3 kernel with a dilation ratio of 2 will have the same **field of view** as a 5x5 kernel, while only using 9 parameters.



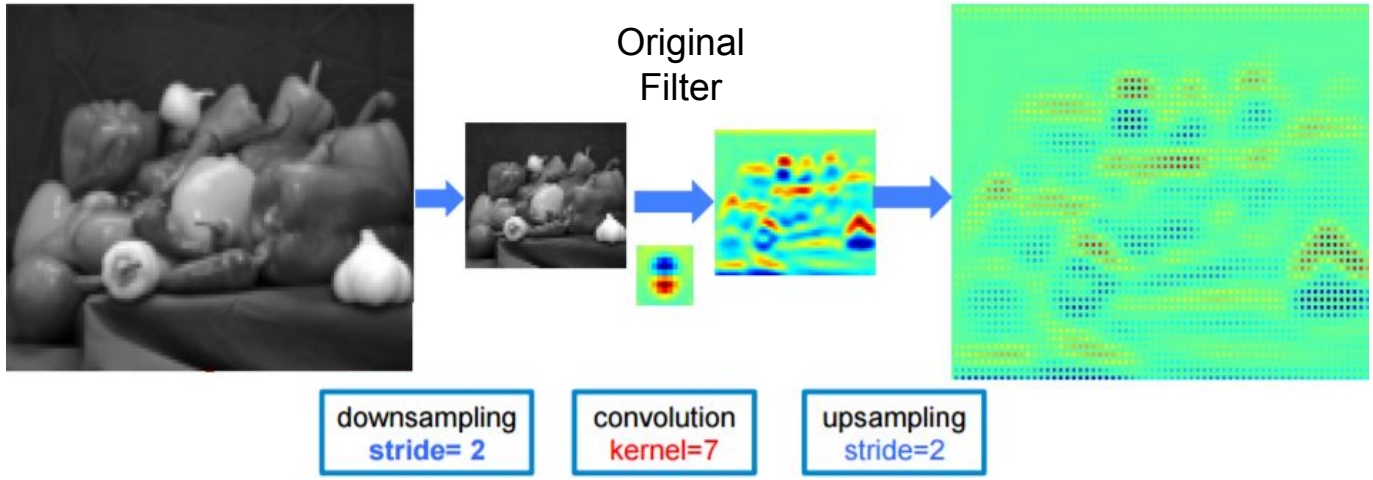
Convolução Dilatada - Padding



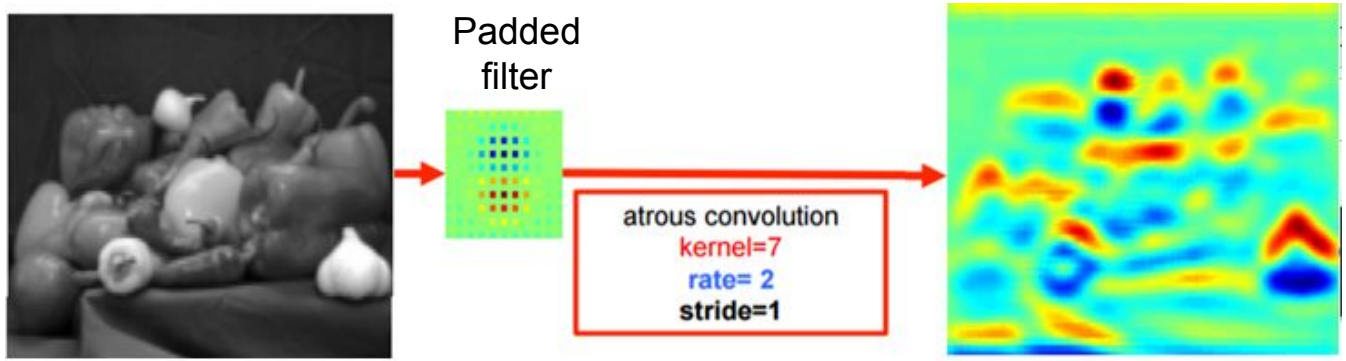
Dilated Convolution

Dilated Convolution

Standard convolution

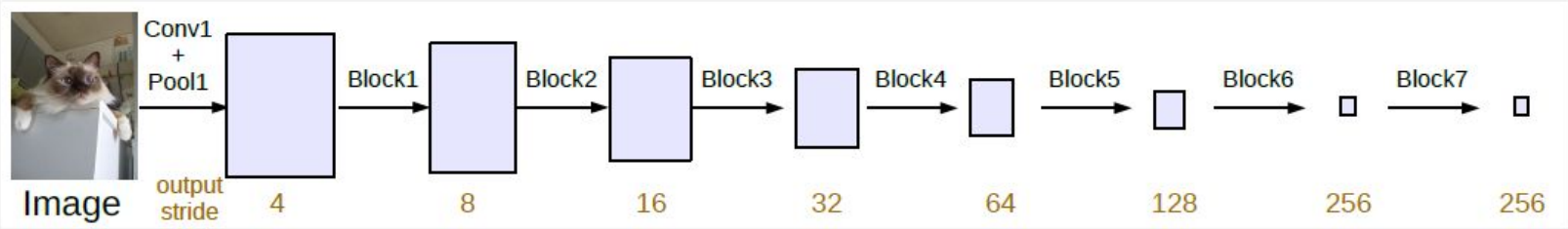


Atrous convolution

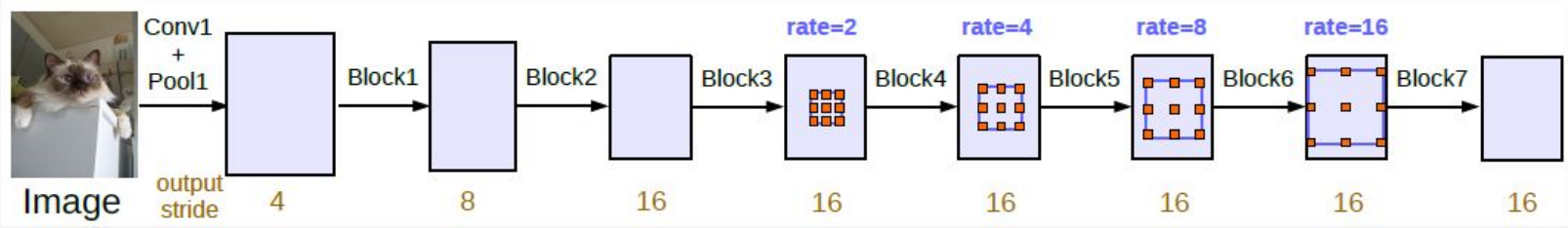


Dilated Convolution

- We can keep the stride constant, but with a larger field-of-view without increasing the number of parameters or the amount of computation.
- We have a larger feature-map, which is good for semantic segmentation.

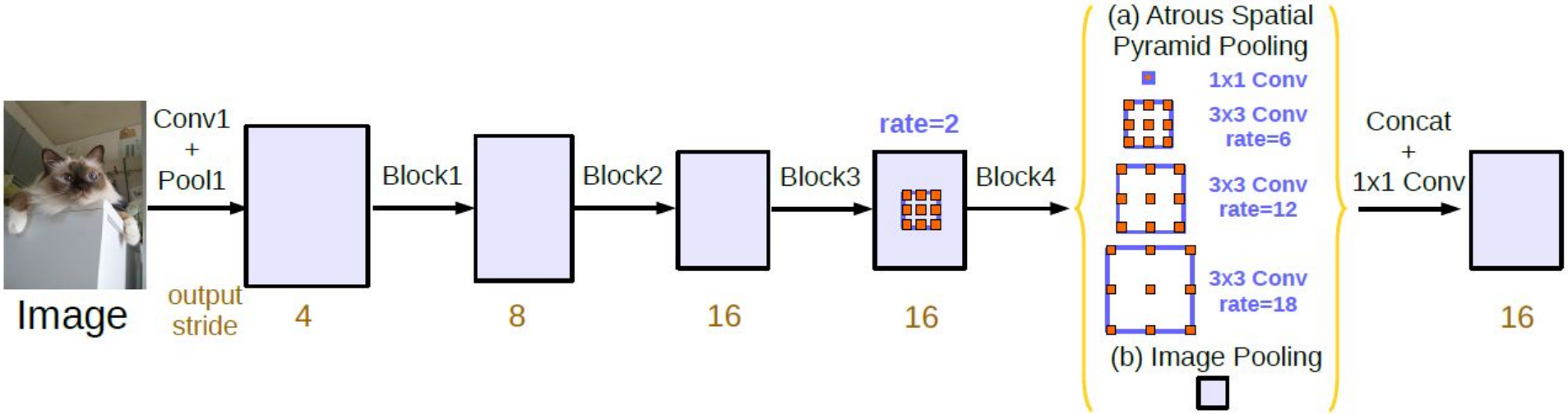


(a) Going deeper without atrous convolution.



(b) Going deeper with atrous convolution. Atrous convolution with $rate > 1$ is applied after block3 when $output\_stride = 16$.

Atrous Spatial Pyramid Pooling (ASPP)



Chen, L. C., Papandreou, G., Schroff, F., & Adam, H. (2017). Rethinking atrous convolution for semantic image segmentation. arXiv preprint arXiv:1706.05587.

Semantic Segmentation - Summary

- Pixel-Wise Classification
  - Inefficient → redundant operations
- Fully Convolutional Networks (FCNs)
  - Drawback: problems with upsampling
  - Advantages: (1) easy to train; (2) backbones can be used.
- Deconvnets
  - Too many parameters to optimize! Hard to converge.
- SegNet
  - Pooling indexes help to converge.
- U-Net
  - Implement skip connections to allow convergence
  - Data augmentation and loss functions more appropriate to biomedical applications
- DeepLab
  - Use dilated convolutions to keep spatial resolution

Semantic Segmentation - Summary

- Pixel-Wise Classification
  - Inefficient → redundant operations
- Fully Convolutional Networks (FCNs)
  - Drawback: problems with upsampling
  - Advantages: (1) easy to train; (2) backbones can be used.
- Deconvnets
  - Too many parameters to optimize! Hard to converge.
- SegNet
  - Pooling indexes help to converge.
- U-Net
  - Implement skip connections to allow convergence
  - Data augmentation and loss functions more appropriate to biomedical applications
- DeepLab
  - Use dilated convolutions to keep spatial resolution